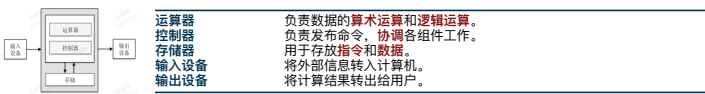


## 第一章 微机概述

**冯诺依曼架构** 现代计算机的理论原型，是后续微机构件的逻辑基础



### 微机设计准则

**准则 1：并行化 (Parallelism)** **并行化**是通过同时处理多个任务或数据来提高计算速度。

|          |                                                                                                                                                                                                                                                       |
|----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 系统级并行    | 在宏观系统层面上的并行。体现为通过 <b>集成多个处理器</b> 、多个磁盘或多通道 I/O 协同工作来提升整体吞吐量与并发能力。<br>指在单个 CPU 内部通过微架构设计实现的并行。                                                                                                                                                         |
| 处理器芯片级并行 | <b>流水线</b><br>将指令执行过程分解为多个阶段，使多条指令在不同阶段重叠执行，提升吞吐率。<br><b>指令级并行</b><br>包括 VLIW (超长指令字) 和 <b>Superscalar (超标量) 技术</b> ，允许在一个时钟周期内发射/执行多条指令。<br><b>SIMD</b><br>单指令多数据：一条指令同时处理多个数据元素 (常用于多媒体/向量计算)。<br><b>多核处理器</b><br>在一片芯片上集成多个执行核心，采用并行线程/进程提升并发处理能力。 |
| 电路级并行    | 在逻辑电路层面的并行化。体现为例如超前进位加法器和多银行存储 (Multiple banks of memory，支持并行读/写以提高内存带宽)。                                                                                                                                                                             |

**准则 2：局部性原理 (Locality)** **统计规律**：程序执行时呈现**90/10 规律**，即 90% 的时间花费在 10% 的指令和数据上，说明工作负载在时间和空间上高度集中。

|       |                                                                                       |
|-------|---------------------------------------------------------------------------------------|
| 时间局部性 | 如果某条指令或数据被访问了，则在不久的将来它很可能会被再次访问。<br>典型场景：循环结构中的 <b>指令</b> 、 <b>循环变量</b> 等频繁重复访问的对象。   |
| 空间局部性 | 如果某条指令或数据被访问了，则与其地址相邻的指令或数据通常也会很快被访问。<br>典型场景：数组元素的 <b>顺序访问</b> 、 <b>顺序执行</b> 的代码指令流。 |

**准则 3：阿姆达尔定律** 注重优化使用频率高的部分，定律说明某一部分的改进对整体性能提升取决于该部件的使用比例与改进幅度。

|                   |                                                                                                               |
|-------------------|---------------------------------------------------------------------------------------------------------------|
| <b>Speedup 定义</b> | 加速比 ( <i>Speedup</i> ) 定义为： $Speedup = \frac{\text{改进前的总执行时间}(ExTime_{old})}{\text{改进后的总执行时间}(ExTime_{new})}$ |
| 核心变量              | $Fraction_{enhanced}$ ：被改进部分在原系统中所占时间比例 (0 ~ 1)。<br>$Speedup_{enhanced}$ ：被改进部分自身的加速倍数。                       |
| 最终公式              | $Speedup = \frac{1}{(1 - Fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}}$              |

● **公式解析**：  $(1 - Fraction_{enhanced})$  为未改进部分的时间比例 (系统的瓶颈)， $\frac{Fraction_{enhanced}}{Speedup_{enhanced}}$

为改进后该部分剩余的时间比例。该公式用于估算在**给定改进范围和幅度下的整体收益**，常用于计算题与性能平衡分析。

|                   |                                                                                                                                                                                              |
|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ● <b>定理 1-1</b> : |                                                                                                                                                                                              |
| 题目条件              | 可优化部分占比 $Fraction_{enhanced} = 0.4$ ;<br>该部分加速倍数 $Speedup_{enhanced} = 10$ .                                                                                                                 |
| 计算过程              | $Speedup = \frac{1}{(1 - Fraction_{enhanced}) + \frac{Fraction_{enhanced}}{Speedup_{enhanced}}} = \frac{1}{(1 - 0.4) + \frac{0.4}{10}} = \frac{1}{0.6 + 0.04} = \frac{1}{0.64} \approx 1.56$ |

**关键结论** 虽然局部提高了 **10 倍**，但由于它只占总运行时间的 **40%**，整体加速比仅约 **1.56**，即未改进部分限制整体性能提升。

|                |                                                                                                                                            |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------|
| <b>处理器性能公式</b> | ● <b>概述</b> : CPU 时间 = 程序的 CPU 时钟周期数 × 时钟周期时间。                                                                                             |
| <b>CPI</b>     | 平均每条指令耗时的时钟周期数, $CPI = \frac{\text{程序运行所需的 CPU 时钟周期总数}}{\text{指令条数 (IC)}}。$                                                                |
| 性能公式           | $CPU\ time = \frac{Instruction\ Count \times CPI \times Clock\ cycle\ time}{\text{或}} = \frac{Instruction\ Count \times CPI}{Clock\ Rate}$ |

**Instruction Count** 由**程序与编译器/指令集架构**决定 (优化编译、指令密度影响指令条数)。  
**CPI** 由**微架构**决定 (流水线、乱序、缓存命中、分支预测等影响每指令平均周期数)。  
**Clock cycle time** 由工艺与微架构时序决定 (**更短的时钟周期**或更高的频率可直接降低 CPU time)。

- **定理 1-2**:
- **题目背景**:
  - 浮点运算 (FP) 指令：出现概率 **25%**，每指令  $CPI = 4$ 。
  - FPSQR (浮点平方根) 指令：出现概率 **2%**，每指令  $CPI = 20$  (耗时很高的特例)。
  - 其它指令：出现概率隐含为  $1 - 25\% = 75\%$ ，每指令  $CPI = 1.33$ 。
- **基线分析 (Baseline)**：系统原始平均 CPI (加权平均)：

$$CPI_{original} = \sum(CPI_i \times Frequency_i)$$

根据简化计算： $CPI_{original} = 4 \times 25\% + 1.33 \times 75\% = 1 + 0.9975 \approx 2。$

- **优化方案**：两种优化方案：
  1. 方案 (1)：针对痛点优化——将 FPSQR 的 CPI 从 **20** 降到 **2** (约 10 倍加速)。
  2. 方案 (2)：针对面向整体优化——将所有 FP 的 CPI 从 **4** 降到 **2.5** (约 1.6 倍加速)。
- **简便差值法**：新 CPI = 原 CPI - (单条指令减少的周期数 × 该指令出现频率)。

方案 (1)——优化 FPSQR

- 单条指令变化： $20 \rightarrow 2$ , 减少  $\Delta CPI = 18$ 。
- 频率：2%。
- 对整体影响： $18 \times 2\% = 0.36$ 。
- 新平均 CPI： $2 - 0.36 = 1.64$ 。

方案 (2)——优化所有 FP

- 单条指令变化： $4 \rightarrow 2.5$ , 减少  $\Delta CPI = 1.5$ 。
- 频率：25%。
- 对整体影响： $1.5 \times 25\% = 0.375$ 。
- 新平均 CPI： $2 - 0.375 = 1.625 \approx 1.62$  (保留两位小数)。

● **结论**：比较两方案得出的新平均 CPI：方案 (1) 为 **1.64**，方案 (2) 为 **1.62**，两方案数值接近，但方案 (2) 在本例中略优。

### 数值单位与运算

**数值单位换算** 常用的集中于 KB 周围的单位 (给存储器章节的计算铺垫)

|              |                                                                                                                              |
|--------------|------------------------------------------------------------------------------------------------------------------------------|
| 位 (bit, b)   | 1 Kb = 1024 bit = $2^{10}$ bit; 1 Mb = 1024 Kb = $2^{20}$ bit; 1 Gb = 1024 Mb = $2^{30}$ bit; 1 Tb = 1024 Gb = $2^{40}$ bit。 |
| 字节 (Byte, B) | 1 Byte = 8 bit, 1 KB = 1024 Byte。                                                                                            |
| 字 (Word)     | 一般情况下为 2 Byte (即 16 bit)。                                                                                                    |

**有符号数的表示与运算** ● **补码**：现代计算机均采用补码。补码便于硬件实现加减法，成为实际工程标准。

- **补码加减法运算规则**：核心思想：**减法变加法**，即  $X - Y$  可转化为  $X + (-Y)$ ，简化硬件设计。
  - 加法： $[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$
  - 减法： $[X - Y]_{\text{补}} = [X]_{\text{补}} - [Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$

符号位参与运算：补码加法时，符号位与数值位同等对待，全部参与二进制加法 (逢二进一)。  
● **补码求负 (变补规则)**：  $[-X]_{\text{补}}$  的求法：将  $[X]_{\text{补}}$  的所有位 (**包括符号位**) **按位取反，再加 1**。即“全字取反加一”。注意：**包括符号位**，不区分符号和数值位。

**有符号溢出** ● **有符号数溢出**：当运算结果超出运算器所能表示的有符号数范围 (例如 8 位有符号数为  $-128$  到  $+127$ ) 时，产生错误结果，称为**溢出**。

**判断方法 1 (基于符号位)** 安全情形：同号相减 (正-正、负-负) 或异号相加 (正 + 负)。  
可能溢出情形：同号相加 (正 + 正、负 + 负) 或异号相减 (等价于同号相加)。  
**判定规则**：**同号相加且结果符号与加数符号相反  $\Rightarrow$  溢出**。

**硬件判定**： $Overflow = C_7 \oplus C_6$ 。当  $C_7 \oplus C_6 = 1$  时为溢出，等于 0 则无溢出。  
 $C_7$  为最高位向更高位的进位 (符号位产生的进位)； $C_6$  为次高位向符号位的进位。

● **定理 1-3**:

**CASE 1：负数 + 负数**  
算式：10110101 (负) + 10001111 (负)  
结果：1 01000100 (最左侧的蓝色 1 为  $C_7$ ；注意 8 位结果的最高位为 0，即表面为正)  
判定分析：逻辑法：负数加负数结果变为正数 (符号位变化)  $\Rightarrow$  **溢出**。  
进位法： $C_7 = 1, C_6 = 0$ , 故  $C_7 \oplus C_6 = 1$ , 判定为溢出。

**CASE 2：正数 + 正数**  
算式：01000010 (正) + 01100011 (正)  
结果：10100101 (8 位结果最高位为 1，即表面为负)  
判定分析：逻辑法：正数加正数结果变为负数 (符号位变化)  $\Rightarrow$  **溢出**。  
进位法： $C_7 = 0, C_6 = 1$ , 故  $C_7 \oplus C_6 = 1$ , 判定为溢出。

**CASE 3：正数 + 负数 (异号相加)**  
算式：01001010 (正) + 11001101 (负)  
结果：1 00001111  
判定分析：逻辑理论，异号相加不会溢出。进位法： $C_7 = 1, C_6 = 1$ , 故  $C_7 \oplus C_6 = 0$ , 判定为无溢出。

● **注**：不能仅看是否产生外溢进位  $C_7$  来判断溢出。即使  $C_7 = 1$ ，只要  $C_6 = 1$ ，则  $C_7 \oplus C_6 = 0$ ，溢出标志为 0 (结果正确)。

**2-10 进制编码** ● **BCD 码 (定义)**：BCD 用 4 位二进制表示 1 位十进制数。4 位可表示  $2^4 = 16$  种状态 (0000~1111)，其中仅用前 10 种 (0000~1001) 表示十进制 0 ~ 9，其余 1010~1111 为非法状态。

**压缩 BCD** **每字节存 2 位十进制数**：字节高 4 位存高位十进制数，低 4 位存低位十进制数。  
例：十进制 (25)<sub>10</sub>  $\Rightarrow 2 = 0010, 5 = 0101$ , 合并为 0010 0101。  
**非压缩 BCD** **每字节存 1 位十进制数**：高 4 位通常为 0000，低 4 位存十进制值 0000~1001。  
例：(25)<sub>10</sub> 需两个字节：2 = 0000 0010, 5 = 0000 0101。

● **BCD 加法与十进制调整**：在计算机中算术单元 (ALU) 按二进制规则加法 (逢 16 进 1)，但 BCD 表示的是十进制 (逢 10 进 1)。因此用二进制加法直接处理 BCD 可能产生错误，必须在加法后进行“十进制调整 (加 6 修正)”以恢复正确的十进制结果。

**调整规则** 对每个 4 位半字节 (nibble) 分别检查并必要时修正：只要满足下列任一条件，就对该 4 位加 0110 (即十进制加 6)：  
1. 该 4 位二进制值  $> 9$  (出现 1010~1111)，例如结果为 1100 (12)，须修正。  
2. 该 4 位在运算时产生了向更高位的进位 (或借位)，即半字节产生进位信号。

1001 (9) + 0001 (1) = 1010 (二进制结果)，需按规则加 0110 修正，得到正确的十进制进位结果。  
二进制半字节是以基数 16 (逢 16 进 1) 工作，而十进制以基数 10 (逢 10 进 1) 工作，两者相差为  $16 - 10 = 6$ ，加 6 是将二进制的“多出的”状态区间跳过，从而把二进制进位校正为十进制进位。

**字符编码** ● **定义**：ASCII (American Standard Code for Information Interchange) 采用 **7 位**二进制编码表示字符。存储时通常占用 1 字节 (8 位)，最高位通常置为 0。

**数字 0~9**  
编码范围：0110 0000 0111 1001 (十六进制 30H~39H)。高 3 位固定为 011 (即十六进制的 3)，低 4 位与 BCD 完全一致，故可通过去掉高 3 位或减 30H 得到对应数值。  
大写字母 A~Z 从 41H 开始递增；小写字母 a~z 从 61H 开始递增。  
**大小写转换** **大小/小写相差 20H**，因此可通过翻转第 6 位实现硬件层面的快速大小写转换。  
**最高位 (奇偶校验)** 标准 ASCII 的第 8 位 (MSB) 通常为 0；在数据传输中常被用作**奇偶校验位**以检测传输错误。

**字母编码**  
**大小写转换**  
**最高位 (奇偶校验)**

### 习题解析

**课后习题** ● **1**：把下列二进制数转换成十进制数、十六进制数及 BCD 码形式。

|     |                |
|-----|----------------|
| (1) | 10110010B      |
| (2) | 010111011.101B |

● **启示意义**：十进制最好解决。16 进制就是 4 位一换，高位不够四位的就补符号位，BCD 码我们就使用最基础的压缩形式，4 位一个十进制数来表示。

● **解**：十进制数、十六进制数及 BCD 码形式如下：

(1) 十进制数：178<sub>10</sub>；十六进制数：B2H；BCD 码 (压缩)：0001 0111 0010。  
(2) 十进制数：93.625<sub>10</sub>；十六进制数：5D.AH；BCD 码 (压缩)：1001 0011 . 0110 0010 0101。

|                             |        |
|-----------------------------|--------|
| ● <b>2</b> ：把下列十进制数转换成二进制数。 |        |
| (1)                         | 100D   |
| (2)                         | 1000D  |
| (3)                         | 67.21D |

● **启示意义**：主要是第三问，令  $r_0$  为该数的小数部分，依次计算  $b_i = \lfloor 2r_{i-1} \rfloor, r_i = 2r_{i-1} - b_i$ ，得到二进制小数为  $0.b_1b_2\dots b_n$ ，当  $r_n = 0$  或达到所需精度时停止。

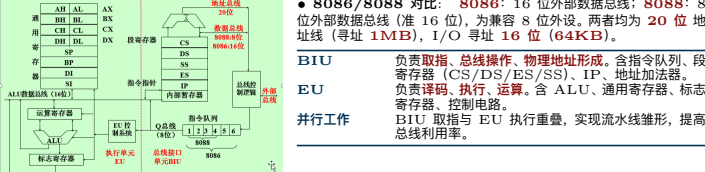
● **解**:

|     |               |
|-----|---------------|
| (1) | 1100100B      |
| (2) | 1111101000B   |
| (3) | 1000111.0011B |

### 考研指导

### 习题指导

## 第二章 微机结构



**总线接口单元 BIU** ● **概述**：BIU 负责执行所有总线周期：**取指** (从存储器取指令并送入指令队列)、**存取数据** (对存储单元或 I/O 端口读/写) 以及总体的总线时序与握手。BIU 产生并响应外部总线控制信号，协调地址/数据/控制总线的时序与驱动。

|               |                                                                                                                                          |
|---------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>功能</b>     | ● <b>取指</b> ：从物理内存按 CS:IP 指定地址取出指令并填入指令队列。<br>● <b>存取数据</b> ：按有效地址执行对内存或 I/O 的读/写总线周期。<br>● <b>执行总线周期</b> ：发起/响应总线信号，处理等待 (READY)、总线仲裁等。 |
| <b>物理地址计算</b> | $PA = \text{段基址} \times 10H + \text{偏移地址}$<br>段基址由 <b>段寄存器</b> (16 位) 左移 4 位提供，偏移地址由 <b>IP</b> 或 <b>EU</b> 计算。                           |

|                  |                                                                         |
|------------------|-------------------------------------------------------------------------|
| <b>组成</b>        | ● <b>组成结构</b> ：BIU 由段寄存器、指令指针、(物理) 地址加法器与总线控制电路组成                       |
| <b>段寄存器</b>      | CS, DS, SS, ES：存放各段的段基址 (段起始地址)。                                        |
| <b>指令指针 (IP)</b> | IP：存放下一条指令在代码段内的偏移 (offset)。                                            |
| <b>20 位地址加法器</b> | 将 16 位段基址左移 4 位并与 16 位偏移相加，生成 20 位物理地址 (8086 内部寄存器为 16 位，外部地址总线为 20 位)。 |
| <b>总线控制电路</b>    | 产生/接收地址总线、数据总线与控制信号，管理总线周期及相关握手。                                        |

**指令队列** ● **概述**：指令队列是 BIU 实现指令预取与流水线行的关键结构，旨在缩短 EU 的取指等待时间。  
EU 从队列头部取出指令执行，BIU 在队尾补入新指令；二者并行工作以提高吞吐量。

|             |                                                                              |
|-------------|------------------------------------------------------------------------------|
| <b>结构</b>   | <b>FIFO (先进先出) 缓冲队列</b> ，按顺序存放已取回的指令字节。                                      |
| <b>容量</b>   | 8086：6 字节；8088：4 字节 (实际能存多少条取决于指令长度 (x86 指令是可变长指令))                          |
| <b>预取机制</b> | 只要总线空闲且队列内有空位，BIU 即从内存取指并在队尾填入指令字节。<br>8086 保持至少空出 2 字节的空间以触发预取；8088 为 1 字节。 |

当 EU 执行改变控制流的指令时，队列中已预取的顺序流指令将不再有效。即**清空 (Flush)**指令队列的全部内容。清空后，BIU 按跳转指令指定的新物理地址**重新取指**，并自该地址开始填满队列以恢复并行取数。

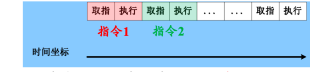
**执行单元 EU** ● **与 BIU 的关系**：EU 不直接连接外部总线，**只从 BIU 的指令队列取指**或通过 BIU 存取数据。EU 计算的是**偏移地址 (有效地址 EA)**，再由 BIU 将 EA 与段基址合成物理地址。

|                    |                                                               |
|--------------------|---------------------------------------------------------------|
| <b>译码与执行</b>       | 接收并译码指令，产生控制信号以执行指令。                                          |
| <b>地址计算</b>        | 完成算术 (加、减、乘、除) 与逻辑 (与、或、非) 运算。<br>计算有效地址 EA 并送 BIU 合成物理地址 PA。 |
| <b>寄存器组成</b>       |                                                               |
| <b>通用寄存器</b>       | AX (累加器), BX (基址), CX (计数), DX (数据/乘除高位)。                     |
| <b>专用/指针/变址寄存器</b> | SP (堆栈指针), BP (基址指针), SI (源变址), DI (目的变址)。                    |
| <b>标志寄存器</b>       | FLAGS：包含状态标志 (如 CF, ZF, OF 等) 与控制标志 (如 IF, DF)，共约 9 位。        |

|                          |                                                                                                                                                                                                      |                                                                                                                                                                  |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>算 术 运 算 单 元 (ALU)</b> | <b>功能</b> ：完成算术与逻辑运算 (16 位)。<br><b>地址计算</b> ：EU 使用 ALU 计算 16 位偏移地址 (EA)，将 EA 送交 BIU；BIU 与段基址合成物理地址， $PA = \text{段基址} \times 10H + \text{偏移地址}$ 。<br><b>要点</b> ：ALU <b>同时负责数据运算与地址运算</b> ，必须与控制器精确同步。 | <b>EU 控制器</b> <b>输入</b> ：从 BIU 指令队列取出的指令字节。<br><b>译码</b> ：识别指令类型与寻址方式，确定操作数据来源/去向。<br><b>控制</b> ：产生时序与控制信号，协调寄存器、ALU 与数据通路执行：取操作数、驱动 ALU、寄回结果、更新标志位并处理队列/跳转相关动作。 |
|--------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**指令流水线** ●概述： 指令流水线是 BIU 与 EU 分离架构在时间维度上的实现：通过指令预取与并行执行来提高总线与执行单元的利用率，缩短执行延迟。

●串行执行模式（8080）：



|      |                                                           |
|------|-----------------------------------------------------------|
| 工作原理 | 取指 (Fetch) 与执行 (Execute) 严格串行：先取指再执行，二者互斥，时间上首尾相接。        |
| 时序示意 | 时序图中 <b>取指块与执行块不重叠</b> ，CPU 在取指期间执行部件空闲；执行期间总线空闲（除非指需访存）。 |
| 缺陷   | 总线利用率低，系统吞吐受取指速度限制。总耗时 = ∑(取指时间 + 执行时间)。                  |

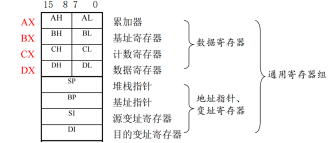
●8086 流水时序图详解：

|             |                                                                                    |
|-------------|------------------------------------------------------------------------------------|
| 初始阶段 (建管线)  | 复位或跳转后队列为空，EU 必须等待 BIU 首次取指，产生流水线建立开销（等待块）。                                        |
| 稳态运行 (并行阶段) | EU 执行指令时，BIU 持续预取并填队列，取指与执行在时间上重叠，队列实现指令的缓冲与解耦。                                    |
| 总线占用 (访问数据) | 若 EU 正在执行需访存/I/O 的指令（例：MOV AX, [BX]），BIU 将中断预取，转去执行数据传输周期（“得到数据”），预取暂停并在数据传输完成后恢复。 |

|          |                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------|
| ●注意事项：   |                                                                                                     |
| 跳转/异常优先级 | 改变控制流（如跳转、异步中断）会清空指令队列（Flush），导致 <b>流水线重建，开销显著</b> 。 <b>数据传输周期相对指令预取有更高优先级</b> ：实现上需处理预取暂停与恢复的状态衔接。 |

**8086 寄存器结构** ●概述：8086 共含 **14** 个寄存器，且均为 **16 位** 宽。

●通用寄存器组（共 8 个）：用于算术/逻辑运算、数据暂存与地址运算。



**数据寄存器** AX, BX, CX, DX，负责运算与数据暂存。  
**指针/变址寄存器** SP, BP, SI, DI，主要保存偏移地址，常用于寻址与堆栈操作。

**通用寄存器组** **数据寄存器**

**高/低位拆分** AX, BX, CX, DX 可作为 **16 位寄存器** 使用，也可拆分为 **两个 8 位寄存器** 使用（高 8 位以 H 结尾，如 AH；低 8 位以 L 结尾，如 AL）。该设计兼容 8 位微处理器并方便处理字节级数据。

●AX (Accumulator, 累加器)：

**主要功能** 算术/逻辑运算的主要操作数寄存器，常用于加减乘除等指令的隐含操作数。  
**隐含用法** I/O 指令交换数据时，IN/OUT 指令通常使用 AX (16 位) 或 AL (8 位) 作为**数据中转寄存器**。

●CX (Count, 计数寄存器)：

**专用功能** 字符串/循环类指令的**隐含计数器**（例如 REP MOVSB, LOOP）。  
**机制** CPU 在执行相关指令时会自动使用并递减 CX，直至 CX=0 停止。

**指针与变址寄存器**

●概述：该组寄存器用于存放偏移地址 (Offset)，参与有效地址 (EA) 与物理地址 (PA) 的计算。均为 **16 位**，不能像数据寄存器那样拆分为两个 8 位寄存器。

**共同要点** 存放偏移地址，参与寻址；默认搭配影响物理地址计算；使用时要注意段寄存器的默认选择（见下）。

|    |                                                                                                                    |
|----|--------------------------------------------------------------------------------------------------------------------|
| SP | <b>始终指向当前栈顶</b> ，用于堆栈操作 (PUSH/POP/CALL/RET 等)。默认与堆栈段 DS 联用，物理地址：PA = CS × 16 + SP。常用于访问 <b>栈帧内的局部变量或参数</b> （非栈数据）。 |
| BP | 虽也可作为基址寄存器，但 BP 默认关联 SS（而 BX 默认关联 DS），故以 BP 作为地址寄存器时默认在堆栈段取数。说实话这个不怎么用过，暂时了解即可吧。                                   |

**段寄存器组** ●概述：BIU 提供 CS, DS, SS, ES 四个段寄存器，采用“**段基址 × 16 + 偏移**”的分段寻址以实现 20 位物理地址 (1MB 寻址空间)。

**原理** 8086 内部寄存器均为 16 位，外部地址为 20 位，故采用段：偏移机制，段基址左移 4 位（等同乘以 16）后加偏移形成物理地址。

●CS —Code Segment:

**功能** 取指公式：物理地址 = 16 × CS + IP (即 CS 左移 4 位再加 IP)。  
**关联** 与 IP 配合，唯一定位下一条要执行的指令。

●常见物理地址组合：

取指 16 × CS + IP  
普通数据 16 × DS + 偏移 (如 [BX])  
串操作源 16 × DS + SI  
串操作目的 16 × ES + DI  
堆栈地址 16 × SS + SP

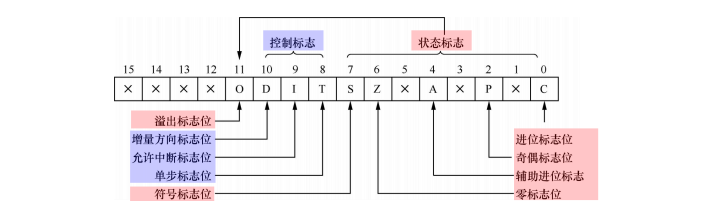
**控制寄存器组** ●概述：控制寄存器组包含 IP (指令指针) 与 FLAGS/PSW (标志寄存器)，用于控制指令流与记录处理器状态。

●IP —指令指针：

**功能** 存放 CPU 下次要取出的指令的偏移地址 (offset)。  
**关联** 与 CS 配合形成物理地址：PA = 16 × CS + IP

**不可直接访问** 用户程序不能用普通数据传送指令 (如 MOV) 修改 IP。其值仅由**转移指令 (JMP)**、**调用指令 (CALL)**、**返回指令 (RET)** 等自动改变。

**处理器状态字 (PSW)**



●总体结构：PSW 是一个 16 位寄存器；实际定义的有效标志位为 **9 个**，其余位为保留（图中以 × 标示，编程时通常忽略）。

**状态标志**：(共 6 个) 反映 **ALU 最近一次运算结果**，用于条件判断 (JZ/JC/JO 等)。

CF **进位标志**，反映无符号算术的进位/借位。  
PF **奇偶校验标志**，反映结果低 8 位中 1 的偶/奇性。  
AF **辅助进位标志**，用于 BCD 调整 (半字进位)。  
ZF **零标志**，结果为 0 时置 1。  
SF **符号标志**，反映结果的符号 (最高位)。  
OF **溢出标志**，反映有符号运算的溢出。

CF **一进位标志** 判定：CF=1 表示无符号加法有进位或减法有借位；CF=0 表示无进位/借位。  
ZF **一零标志** 判定：CF=1 且仅当  $C_n \oplus SF=1$  结果为 0；ZF=0 表示结果非 0。  
AF **一辅助进位** 判定：记录低半字节 (D3) 向高半字节 (D4) 的进位/借位，主要用于 **BCD 调整**（如 DAA 指令），这个应该不考虑。  
PF **一奇偶标志** 判定：仅检查结果的**低 8 位**：PF=1 表示低 8 位中含偶数个 '1'；常用于奇偶校验场景。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

**CF=1** 表示无符号加法有进位或减法有借位；**CF=0** 表示无进位/借位。  
**CF=0** 否则。

●DS / SS:

DS 存放全局数据/变量所在的段基址。默认用于普通数据寻址；  
SS 存放堆栈段基址；堆栈操作强制使用 SS 与 SP。物理地址 = 16 × SS + SP。**LIFO 原则**适用。

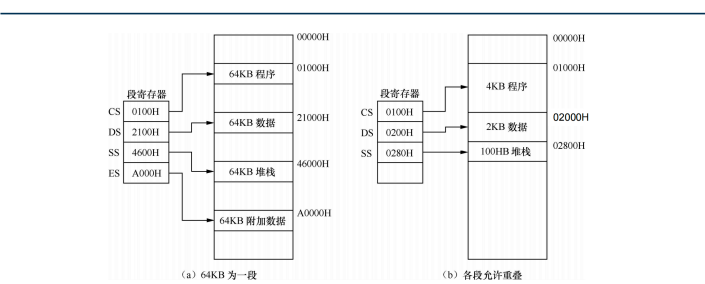
●ES —Extra Segment:

**功能** 辅助数据段，用于字符串操作的目的 (与 DI 配合)。  
**初始化** DS 与 ES **需由程序员显式设置**；若相等则两段地址重合。

与 FLAGS/PSW (标志寄存器)，用于控制指令流与记录处理器状态。

●PSW/FLAGS —标志寄存器：

**结构** 16 位寄存器，常用有效位为 9 位 (状态标志 + 控制标志)。  
**状态标志 (6 位)** 记录上一次 ALU 运算的结果特征，例如 ZF (零标志)、CF (进位)、OF (溢出) 等，**条件转移指令 (如 JE/JZ)** 判断。  
**控制标志 (3 位)** 用于控制 CPU 行为，例如 IF (中断允许)、DF (方向标志)、TF (陷阱标志，这个少见)。



(a) 64KB 为一段 (b) 各段允许重叠

**独立分段结构** 代码段 (CS)、数据段 (DS)、堆栈段 (SS) 互不重叠，各自占据不同的 64KB 区域。  
**含义** 段是逻辑划分，内存仍为线性空间，段边界由段寄存器值决定。

●段长度灵活：实际段长度可小于 64KB（例如 4KB、2KB），未使用空间可被其他段复用或重叠使用。

**20 位物理地址组成** 逻辑地址：程序段地址 (CS)、数据段地址 (DS)、堆栈段地址 (SS) 互不重叠，各自占据不同的 64KB 区域。  
**物理地址** 段地址 × 16H + 偏移地址 (如 CS:IP)。

●20 位物理地址：在地址总线上传输的唯一 20 位地址，范围 00000H ~ FFFFFH。  
**逻辑地址** 程序段地址 (CS)、数据段地址 (DS)、堆栈段地址 (SS) 互不重叠，各自占据不同的 64KB 区域。  
**计算公式** 物理地址 = 段地址 × 16H + 偏移地址

**多对一映射** 同一物理单元可由不同的逻辑地址 (不同的段：偏移组合) 表示；即**存在多组段：偏移映射到相同的 20 位物理地址 (即上文提到的可重叠分段机制)**。

●例 1 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 2 (验证)： 9002:1320 = 90020H + 01320H = 91340H  
9002:1320 = 90020H + 01320H = 91340H  
9002:1320 = 90020H + 01320H = 91340H  
9002:1320 = 90020H + 01320H = 91340H

●例 3 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 4 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 5 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 6 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 7 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 8 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 9 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 10 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 11 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 12 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 13 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 14 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 15 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 16 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 17 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 18 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 19 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 20 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 21 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 22 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 23 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 24 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H

●例 25 (验证)： 9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H  
9004:1300 = 90040H + 01300H = 91340H









|                      |                                                                                                                     |
|----------------------|---------------------------------------------------------------------------------------------------------------------|
| <b>● AND 一清零/屏蔽：</b> |                                                                                                                     |
| <b>原理</b>            | <span><span>    x ∧<!-- ∧ --> 0 = 0 , x ∧<!-- ∧ --> 1 = x   {\displaystyle x\wedge 0=0,x\wedge 1=x}  </span></span> |
| <b>示例</b>            | AND AL, 0FH （0FH = 0000 1111B）                                                                                      |
| <b>效果</b>            | 高 4 位清零，低 4 位保持                                                                                                     |
| <b>● TEST 一状态检测：</b> |                                                                                                                     |
| <b>原理</b>            | 执行 AND 但不保存，仅更新 ZF                                                                                                  |
| <b>示例</b>            | TEST AL, 80H; JNZ NEXT                                                                                              |
| <b>效果</b>            | 检测 AL 最高位：为 1 则 ZF=0 跳转                                                                                             |

|                     |           |
|---------------------|-----------|
| <b>● NOT 一全域取反：</b> |           |
| <b>示例</b>           | NOT AL    |
| <b>效果</b>           | 每位翻转（0↔1） |

|                                                                                                                                                                                |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <b>移位指令</b>                                                                                                                                                                    |  |
| <b>● 指令格式：</b> <span><span> </span></span> 格式：Mnemonic Destination, Count。 <span><span> </span></span> Destination：通用寄存器或存储器； <span><span> </span></span> Count：立即数 n 或寄存器 CL。 |  |

|                    |                                                                                                                                                                                                                                                           |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>8086 移位次数限制</b> | <span><span>    n = 1 : 可 直 接 写 立 即 数 （ 如 SHL AL, 1 ） .   n &gt; 1 : 必 须 将 次 数 存 入 CL, 然 后 使 用 CL 作 为 操 作 数 （ 不 可 直 接 写 立 即 数 或 使 用 其 他 寄 存 器 ） .   {\displaystyle n=1:可直接写立即数（如SHLAL,1）.n&gt;1:必须将次数存入CL,然后使用CL作为操作数（不可直接写立即数或使用其他寄存器）.}  </span></span> |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|           |                                                          |                                                                 |
|-----------|----------------------------------------------------------|-----------------------------------------------------------------|
| 80286+ 系统 | 解除限制，允许 $n$ 为任意立即数。                                      |                                                                 |
|           | <b>逻辑移位</b>                                              | <b>算术移位</b>                                                     |
|           | <b>SHL</b><br>逻辑左移。MSB $\rightarrow$ CF, LSB 补 0。        | <b>SAL</b><br>算术左移。与 SHL 作用相同。                                  |
|           | <b>SHR</b><br>逻辑右移。LSB $\rightarrow$ CF, MSB 补 0 (无符号数)。 | <b>SAR</b><br>算术右移。LSB $\rightarrow$ CF, MSB 保持原值进行符号扩展 (带符号数)。 |

- 标志位影响：** CF：存放被移出的最后一位。 OF：仅当移位次数为 1 时定义（**检测是否改变符号**），n>1 时未定义。 SF, ZF, PF：按移位后的结果更新。

|                                                            |                                                               |
|------------------------------------------------------------|---------------------------------------------------------------|
| <b>移位后的标志位</b>                                             | <b>移位与算术运算的对应</b>                                             |
| <b>CF</b> (进位标志) 保存最后被移出的一位。无论左移还是右移，志) 被挤出的位都进入 CF。       | <b>无符号数</b> <span><span> </span></span> SHL ×2; SHR ÷2。       |
| <b>OF</b> (溢出标志) <b>仅在移位次数为 1 时有效</b> 。当移位次数 > 1 时，OF 未定义。 | <b>有符号数</b> <span><span> </span></span> SAL ×2; SAR ÷2（保持符号）。 |

- OF 判定规则（移位次数 ==1）：** OF = *CF* ⊕ MSB<sub>后</sub>（移位后的 CF 与最高位 XOR）。

对**左移（SHL/SAL）**：若符号位改变，则 OF=1（溢出）；否则 OF=0。

对**算术右移（SAR）**： 通常 OF=0，符号位不变。

对**逻辑右移（SHR）**：若移位前最高位为 1，则 OF=1。

- 其他标志：** SF, ZF, PF 按结果正常更新；AF 未定义。

|                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------|
| <div><div><div><div><span></span></div><div><span></span></div></div><div><div><span></span></div><div><span></span></div></div></div></div> |
|----------------------------------------------------------------------------------------------------------------------------------------------|

- 应用场景：** 循环移位常用于**轮询测试定位、数据格式转换**（如 BCD ↔ Binary）等需要逐位处理的场景。

- 标志位特性：** 循环移位指令不影响 SF、ZF、AF、PF，仅修改 CF 和 OF。

|    |                                        |
|----|----------------------------------------|
| CF | 存放最后被移出的一位。                            |
| OF | 仅在移位次数为 1 时定义（判断是否改变符号），移位次数 > 1 时未定义。 |

- 应用：多字节长移位：** RCL/RCR 的主要用途是实现跨越多个寄存器的整体移位。例如对 32 位数据（存放在 DX:AX，DX 为高位）进行左移：

|           |                                                      |                                                                                                                                                                                                                                                  |
|-----------|------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SHL AX, 1 | 低 16 位左移，AX 最高位移出进入 CF，最低位补 0。                       | <div> <ul style="list-style-type: none"><li><b>● 结果：</b><span><span> </span></span>实现了 DX:AX 作为一个整体的 32 位左移，数据通过 CF 在两个寄存器间实现无缝传递。</li> <li><b>● 原理：</b><span><span> </span></span>CF 充当“中间渡口”，将低位寄存器的溢出位精确传递至高位寄存器的低端，确保数据连续性。</li></ul> </div> |
| RCL DX, 1 | 高 16 位带进位左移。CF（即前面从 AX 移出的位）移入 DX 最低位，DX 最高位移出进入 CF。 |                                                                                                                                                                                                                                                  |

|                 |                                                     |
|-----------------|-----------------------------------------------------|
| <b>字符串操作类指令</b> | 不同于普通 MOV 指令可以随意指定寄存器，字符串指令对源地址、目的地址、使用的寄存器有着严格的规定。 |
| <b>概述</b>       |                                                     |

|               |                                           |
|---------------|-------------------------------------------|
| <b>传送类</b>    | MOVS (Move String)                        |
| <b>存取类</b>    | LODS (Load String), STOS (Store String)   |
| <b>比较/扫描类</b> | CMPS (Compare String), SCAS (Scan String) |
| <b>I/O 类</b>  | INS, OUTS                                 |

- 共性：** 字符串指令执行时，CPU 自动从特定寄存器读取地址并可自动更新索引寄存器。以下左右并列展示源/目的的对称规则。

|              |                                 |              |                            |
|--------------|---------------------------------|--------------|----------------------------|
| <b>角色</b>    | 源字符串 (Source)                   | <b>角色</b>    | 目标字符串 (Destination)        |
| <b>默认段基址</b> | DS                              | <b>默认段基址</b> | ES                         |
| <b>偏移寄存器</b> | SI (或 ESI)                      | <b>偏移寄存器</b> | DI (或 EDI)                 |
| <b>特殊性</b>   | 允许段超越 (可用段前缀覆盖，例：CS:, SS:, ES:) | <b>特殊性</b>   | 不允许段超越：目的串必须位于 ES 段 (硬件规定) |

|                    |                                                  |
|--------------------|--------------------------------------------------|
| <b>● 地址自动修改机制：</b> |                                                  |
| <b>自动更新</b>        | 执行一次字符串指令后，SI 与 DI 自动更新，指向下一单元。                  |
| <b>方向控制</b>        | 由方向标志 DF 控制：DF=0（由 CLD 设）→ 增量；DF=1（由 STD 设）→ 减量。 |
| <b>步长控制</b>        | 与操作宽度相关：字节：±1；字（Word）：±2；双字（Dword）：±4。           |

|                      |                                                                                 |
|----------------------|---------------------------------------------------------------------------------|
| <b>● 字符串指令格式与后缀：</b> |                                                                                 |
| <b>格式一（带操作数）</b>     | 例如：MOVSB dest, src。此类书写只是供汇编器识别， <b>实际寻址由 SI/DI 锁定</b> ，操作数 <i>字节</i> 不改变寻址寄存器。 |
| <b>格式二（助记符后缀）</b>    | 使用后缀指明操作宽度：MOVSB, MOVSW, MOVSD。更直观且常用。                                          |

|             |                                       |
|-------------|---------------------------------------|
| <b>后缀含义</b> | B：字节；W：字 (Word)；D：双字 (Dword, 80386+)。 |
| <b>自动步长</b> | 根据操作宽度自动更新索引寄存器：字节 ±1、字 ±2、双字 ±4。     |

- 重复前缀 (REP 系列)：**

|                        |                                                                   |
|------------------------|-------------------------------------------------------------------|
| REP                    | 先判后执：REP 执行前检查 CX (或 ECX)，若 CX=0 则不执行；否则重复字符串指令并在每次后递减计数器直至 CX=0。 |
| REPE/REPZ, REPNE/REPNZ | 结合 ZF 条件停止：在计数耗尽前若条件不满足则提前退出。                                     |

|              |                                                                                                                                                                |
|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MOVS</b>  |                                                                                                                                                                |
| <b>● 概述：</b> |                                                                                                                                                                |
| <b>功能</b>    | 将内存中一单位的数据从源复制到目的： <span><span>    E S : [ D I / E D I ] ←<!-- ← --> D S : [ S I / E S I ]   {\displaystyle ES:[DI/EDI]\leftarrow DS:[SI/ESI]}  </span></span> |
| <b>要点</b>    | MOVS 是 <b>内存 → 内存专用的搬运指令</b> ；普通 MOV 无法写成 MOV [DI],[SI]。                                                                                                       |

|                     |                                        |
|---------------------|----------------------------------------|
| <b>● 后缀（操作宽度）：</b>  | <b>● 指针（SI/DI）自动更新：</b>                |
| MOVSB 传送 1 字节       | <b>DF = 0</b> SI, DI 分别加上 1/2/4（与后缀对应） |
| MOVSW 传送 1 字（2 字节）  | <b>DF = 1</b> SI, DI 分别减去 1/2/4        |
| MOVSD 传送 1 双字（4 字节） |                                        |

- 执行细节（微操作）：** 执行一条 MOVS 的典型步骤：

|             |                                    |
|-------------|------------------------------------|
| <b>读</b>    | 从 DS <span> </span> : SI 读取 1 单位数据 |
| <b>写</b>    | 将该数据写入 ES <span> </span> : DI      |
| <b>指针更新</b> | 根据 DF 与操作宽度调整 SI/DI                |

- 与 REP 配合：**

|             |                                  |
|-------------|----------------------------------|
| <b>步骤 1</b> | 检查 CX = 0。若为 0，立即不执行串操作并跳到下一条指令。 |
| <b>步骤 2</b> | 若 CX ≠ 0，执行 CX ← CX − 1。         |
| <b>步骤 3</b> | 执行一次串操作指令（例如 MOVSB）。             |
| <b>步骤 4</b> | 返回步骤 1，直至 CX = 0。                |

|                      |                                                            |
|----------------------|------------------------------------------------------------|
| <b>示例（作业 2-3 答案）</b> | LEA SI, SOURCE<br>LEA DI, DEST<br>MOV CX, 100<br>REP MOVSB |
|----------------------|------------------------------------------------------------|

|                                                                                    |                                                                                    |
|------------------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>存取类指令：</b> LODS / STOS                                                          | <b>● STOS —Store String:</b>                                                       |
| <b>● LODS —Load String:</b>                                                        | <b>功能</b> <span><span> </span></span> 将累加器中的值写入目的内存单元。                             |
| <b>功能</b> <span><span> </span></span> 将内存中源字符串的当前元素装入累加器。                          | <b>指令形式</b> <span><span> </span></span> STOSB: ES:[DI] ← AL<br>STOSW: ES:[DI] ← AX |
| <b>指令形式</b> <span><span> </span></span> LODSB: AL ← DS:[SI]<br>LODSW: AX ← DS:[SI] | <b>指针更新</b> <span><span> </span></span> DI 根据操作宽度自动加/减：1/2/4（字节/字/双字）。             |
| <b>指针更新</b> <span><span> </span></span> SI 根据操作宽度自动加/减：1/2/4（字节/字/双字）。             | <b>典型用途</b> <span><span> </span></span> 常与 REP 前缀配合用于内存初始化（高频考点）。                  |
| <b>考试注意</b> <span><span> </span></span> REP LODS <b>很少使用——每次一味覆盖累加器</b>            |                                                                                    |

|                    |                                   |
|--------------------|-----------------------------------|
| <b>示例（内存清零）</b>    | MOV AL, 0  MOV CX, 100  REP STOSB |
|                    | 作用：将 ES:[DI] 起始的 100 个字节全部置零。     |
| <b>比较类指令：</b> CMPS |                                   |

|             |                                                            |
|-------------|------------------------------------------------------------|
| <b>操作</b>   | 对两字符串单元做减法比较，通常记作 CMP ES:[DI], DS:[SI] 即“ <b>目的 - 源</b> ”。 |
| <b>结果</b>   | 仅更新标志位：AF, CF, OF, PF, SF, ZF。内存内容保持不变。                    |
| <b>指针更新</b> | SI 与 DI 同步按操作宽度（B/W/D）自动增/减；CMPS 是唯一同时修改两指针的字符串指令。         |

- 与 REPE/REPZ 的配合：**

|             |                                                                                 |
|-------------|---------------------------------------------------------------------------------|
| <b>循环条件</b> | REPE/REPZ 在执行 CMPSB/W/D 时重复的条件为： <i>CX ≠ 0</i> 且 <i>ZF = 1</i> （即“未比完且当前字符相等”）。 |
| <b>应用</b>   | 常用 REPE CMPSB 或 REPE CMPSW 来判定两字符串是否相等。                                         |

|                      |                                                   |
|----------------------|---------------------------------------------------|
| <b>字符串扫描指令：</b> SCAS |                                                   |
| <b>● 概述：</b>         |                                                   |
| <b>比较对象</b>          | <b>AL/AX/EAX 与目的字符串</b> ES <span> </span> : [DI]。 |
| <b>操作</b>            | 执行 AL/AX/EAX − ES <span> </span> : [DI]。          |
| <b>结果</b>            | 不保存被比较的数据， <b>仅影响标志位（ZF/CF/OF 等）</b> 。            |
| <b>指针更新</b>          | 仅修改 DI（按操作宽度自动增/减，受方向标志 DF 控制：字节 ±1、字 ±2、双字 ±4）。  |

- 与 REPNE/REPNZ 的配合：** 在内存中搜索与累加器寄存器中匹配或者不匹配的字符串。

|                                                                                                                    |                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------|
| REPNE/REPNZ 重 复 扫 描， <b>先判后执</b> ：在每次迭代前检查 <i>CX ≠ 0</i> ；若 <i>CX = 0</i> 则结束循环，否则执行一次 SCASx 并根据 <i>ZF</i> 决定是否继续。 | <b>使用要点</b> <span><span> </span></span> 使用前应将搜索关键字置入 AL/AX 并初始化 <i>CX</i> （为需扫描的长度）。扫描结束后用 <i>ZF</i> 判定是否因匹配而退出；DI 若找到则指向匹配字符的下一字节（位置为目标后缀）。 |
| <b>循环条件</b>                                                                                                        | 在循环继续时时满足： <i>CX ≠ 0</i> 且 <i>ZF = 0</i> （当前字节/字与累加器不等）。含义：只要未找到匹配项就继续扫描。                                                                    |

- 示例：** 假设已将字符串长度置入 CX，关键字置入 AL：

|             |                                                                                      |
|-------------|--------------------------------------------------------------------------------------|
| <b>示例代码</b> | MOV AL, 0FFH  REPNE SCASB                                                            |
| <b>结果判定</b> | 随后执行 JNE ERR: 如果 ZF=0（JNE 成立）则表示循环因 <i>CX = 0</i> 而结束（未找到）；若 ZF=1 则表示找到匹配项（JNE 不跳转）。 |
| <b>指针位置</b> | 若找到，DI 指向目标字符的下一位；若未找到，DI 通常指向扫描末尾后的下一位（取决于初始 <i>DI/CX</i> ）。                        |

|                                          |                                                        |                                                        |
|------------------------------------------|--------------------------------------------------------|--------------------------------------------------------|
| <b>重复前缀（REP 系列）总结</b>                    |                                                        |                                                        |
| REP（无条件重复）                               | REPE / REPZ（相等/为零）                                     | REPNE / REPNZ（不相等/不为零）                                 |
| <b>适用指</b> MOVSB, STOS（内存搬运/初始化）。        | <b>适用指</b> CMPS（比较类）。                                  | <b>适用指</b> SCAS（扫描/搜索）。                                |
| <b>循环逻辑</b> 继续条件： <i>CX ≠ 0</i> 。        | <b>循环逻辑</b> 继续条件（与关系）： <i>CX ≠ 0</i> 且 <i>ZF = 1</i> 。 | <b>循环逻辑</b> 继续条件（与关系）： <i>CX ≠ 0</i> 且 <i>ZF = 0</i> 。 |
| <b>退出条件</b> 仅当 <i>CX = 0</i> 。           | <b>退出条件</b> 任一满足即退出： <i>CX = 0</i> 或 <i>ZF = 0</i> 。   | <b>退出条件</b> 任一满足即退出： <i>CX = 0</i> 或 <i>ZF = 1</i> 。   |
| <b>典型用</b> 内存拷贝或填充：REP MOVSB, REP STOSB。 | <b>典型用</b> 判断两个字符串是否完全相同：REPE CMPSB。                   | <b>典型用</b> 在内存中搜索某值：REPNE SCASB（未找到或找到即停）。             |

|                                                                                                                                                        |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <b>控制转移类指令 分类</b>                                                                                                                                      |  |
| <b>● 概述：</b> <span><span> </span></span> CPU 在正常情况下顺序执行指令（CS:IP 自动增加）。控制转移指令通过 <b>修改 IP/EIP（必要时并修改 CS）来改变执行流</b> 。改变控制流会 <b>导致指令队列清空</b> ，需注意力流水线重建清空。 |  |

|                      |                                                                                            |
|----------------------|--------------------------------------------------------------------------------------------|
| <b>无条件转移（JMP）</b>    | 不检查标志，执行即跳转。可为短/近/远跳转（相对或绝对寻址）。                                                            |
| <b>条件转移（Jcc）</b>     | 根据标志位（如 ZF, CF, OF, SF）决定是否跳转。（例：JE/JZ, JNE/JNZ, JC/JB, JO, JS）。                           |
| <b>循环指令（LOOP 系列）</b> | 以 CX/ECX 作为计数器（例：LOOP, LOOPE/LOOPZ, LOOPNE/LOOPNZ）， <b>每次执行时计数器减 1</b> ，按计数和（可选的）标志决定是否跳转。 |

**调用/返回指令** CALL 将返回地址压栈（**近调用仅压 IP/EIP, 远调用压 CS:IP**）；RET 从栈弹出返回地址并返回（RET n 可顺带清栈）；远返回使用 RETF。

**中断指令（系统级控制）** INT n 为**软件中断**（系统调用/异常触发，CPU 将保存 FLAGS/CS:IP 并跳转到向量表）；IRET 恢复 IP/CS/FLAGS 以返回中断前状态。

|                    |                                                                                       |             |                                                                                                   |
|--------------------|---------------------------------------------------------------------------------------|-------------|---------------------------------------------------------------------------------------------------|
| <b>无条件跳转指令 JMP</b> |                                                                                       |             |                                                                                                   |
| <b>段内转移</b>        | <b>定义：</b> 目标在当前代码段内。<br><b>机制：</b> CS 保持不变，仅修改 IP/EIP。<br><b>要点：</b> 通常不需段前缀或段更改说明符。 | <b>段间转移</b> | <b>定义：</b> 目标在另一个代码段。<br><b>机制：</b> 同时修改 CS 与 IP/EIP（赋 CS:IP）。<br><b>要点：</b> 必须更新 CS 指向新段（远跳/跨段）。 |

|                       |                                                                                     |
|-----------------------|-------------------------------------------------------------------------------------|
| <b>直接转移（Direct）</b>   | <b>目标地址（标号或立即数）直接编码在指令中</b> ，如 JMP label。段内直接只写 IP；段间直接写 CS:IP（近/远跳区分）。             |
| <b>间接转移（Indirect）</b> | <b>目标地址存于寄存器或存储器</b> （例如 JMP [mem] / JMP reg）。可用于段内或段间跳转；若从存储器读取 CS:IP，需明确数据宽度（见下）。 |

- 段内转移—SHORT / NEAR:** 把标签 (label) 的 IP 赋值给 IP。

|                   |                                                         |                  |                                                                                 |
|-------------------|---------------------------------------------------------|------------------|---------------------------------------------------------------------------------|
| <b>SHORT (短程)</b> | <b>格式：</b> JMP SHORT label。<br>8 位带符号位移；范围：−128 ~ +127。 | <b>NEAR (近程)</b> | <b>格式：</b> JMP NEAR label。<br>16 位位移；范围：−2 <sup>15</sup> ~ 2 <sup>15</sup> − 1。 |
|-------------------|---------------------------------------------------------|------------------|---------------------------------------------------------------------------------|

|             |              |                        |
|-------------|--------------|------------------------|
| 1000H:0100H | JMP          | <b>NEXT；(IP)←1130H</b> |
| 1000H:0103H | .....        |                        |
| .....       |              |                        |
| 1000H:0130H | <b>NEXT；</b> | .....                  |

- 示例：** JMP SHORT NEXT JMP NEAR NEXT

|                       |                                                                                                                                    |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <b>间接转移（Indirect）</b> | <b>格式示例：</b> JMP reg、JMP [mem]。<br>操作数宽度（8086）：16 位（从寄存器或内存取出的 16 位作为新的 IP）。<br>注意：若为内存操作数（如 JMP [BX]），则内存中取出的 16 位仅赋给 IP，CS 保持不变。 |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------|

|                            |                                             |                                           |
|----------------------------|---------------------------------------------|-------------------------------------------|
| JMP                        | BX                                          | ；寄存器寻址，(IP)←BX的值                          |
| JMP                        | [BX]                                        | ；寄存器间接寻址，(IP)←DS:[BX]                     |
| <b>● 示例：</b>               | JMP BX <span><span> </span></span> JMP [BX] |                                           |
| <b>● 段间直接转移（Far Jump）：</b> |                                             |                                           |
| 1000H:0100H                | JMP                                         | <b>FAR PTR NEXT；(CS)←2000H，(IP)←0130H</b> |
| .....                      | .....                                       |                                           |
| 2000H:0130H                | <b>NEXT；</b>                                | .....                                     |

|           |                                                                                              |
|-----------|----------------------------------------------------------------------------------------------|
| 格式        | JMP FAR PTR 标号。 FAR PTR 为关键字，强制跨段。                                                           |
| 机制        | 同时修改段寄存器和值：CS ← 目标段，IP/EIP ← 目标偏移。例：当前 1000H:0100H，目标 2000H:0130H，执行后 CS = 2000H，IP = 0130H。 |
| ● 段间间接转移： |                                                                                              |
| JMP       | DWORD PTR [BX] : (IP)←WORD PTR [BX]<br>(CS)←WORD PTR [BX+2]                                  |

|               |                                                                                 |
|---------------|---------------------------------------------------------------------------------|
| <b>存储器操作数</b> | 内存中必须 <b>连续存放目标指针</b> （低地址处为偏移/低位，随后为段/高位）。                                     |
| <b>8086</b>   | 操作数长度为 4 字节（DWORD）：低 16 位 → IP，高 16 位 → CS。                                     |
| <b>类型说明</b>   | <b>必须使用 DWORD PTR (8086)</b> ，因为每个地址都是 2 字节长，段间跳转需 4 字节，所以要从 BX 读 IP，BX+2 读 CS。 |

|                                                                                                                                                                                     |  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <b>条件跳转指令 Jcc</b>                                                                                                                                                                   |  |
| <b>● 概述：</b> <span><span> </span></span> 指令格式：Jcc label (cc 为具体条件)。满足条件时修改 IP/EIP 跳转到标号处执行，否则继续顺序执行。判断依据通常来自 CMP（本质为减法）的结果；条件转移指令本身 <b>不修改</b> 标志寄存器，因而可 <b>连续依据同一次比较使用多个 Jcc</b> 。 |  |

| 无符号数比较（Unsigned）                                                                                                                                                                             | 有符号数比较（Signed）                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| JA / JNBE 条件 $CF = 0$ 且 $ZF = 0$ （严格大于）。<br>JAE / JNB / 条件 $CF = 0$ （不产生借位， $\geq$ ）。<br>JNC<br>JB / JNAE / 条件 $CF = 1$ （产生借位， $<$ ）。<br>JBC<br>JBE / JNA 条件 $CF = 1$ 或 $ZF = 1$ （ $\leq$ ）。 | JG / JNLE 条件 $(SF \oplus OF) = 0$ 且 $ZF = 0$ （严格大于）。<br>JGE / JNL 条件 $(SF \oplus OF) = 0$ （ $\geq$ ）。<br>JL / JNGE 条件 $(SF \oplus OF) = 1$ （ $<$ ）。<br>JLE / JNG 条件 $(SF \oplus OF) = 1$ 或 $ZF = 1$ （ $\leq$ ）。 |

|                        |                                                                                                                                                                         |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 相等/零<br>溢出<br>奇偶<br>符号 | JE / JZ: $ZF = 1$ ； JNE / JNZ: $ZF = 0$ 。<br>JO: $OF = 1$ ； JNO: $OF = 0$ 。<br>JP / JPE: $PF = 1$ （偶校验）； JNP / JPO: $PF = 0$ 。<br>JS: $SF = 1$ （负）； JNS: $SF = 0$ （非负）。 |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

|             |                                                                                           |
|-------------|-------------------------------------------------------------------------------------------|
| 计数器判断（特殊指令） | JCXZ: 当 $CX = 0$ 时跳转； JECXZ (80386+)：当 $ECX = 0$ 时跳转。 <b>这类指令不检查 <math>FLAGS</math></b> 。 |
| 跳转范围        | 8086: 条件跳转通常为短程（Short，8 位有符号位移                                                            |

| 循环指令 LOOP                                                                                                                                                                                                            |  |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|
| <div> <div> <div>LOOP AGAIN</div> <div>↔</div> <div>DEC CX/ECX<br/>JNZ AGAIN</div> </div> <div> <div>等价</div> <div>↔</div> <div>（对标志寄存器<b>不</b>产生影响）</div> <div>↔</div> <div>（对标志寄存器<b>产生</b>影响）</div> </div> </div> |  |

|                |                                                                                                                                                |
|----------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>● 要点提醒：</b> |                                                                                                                                                |
| 跳转距离<br>基本语义   | <b>短程转移</b> （SHORT，8 位带符号位移，范围 $-128 \sim +127$ ）。<br>执行到 LOOP label 时：先 $CX \leftarrow CX - 1$ ，再判断 CX 是否为 0；若 $CX \neq 0$ 则跳转到 label，否则顺序执行。 |
| 标志影响           | LOOP <b>本身不修改任何标志位</b> （ZF/SF/OF/CF 等保持原状）。                                                                                                    |

|          |                                                                                   |            |                                                                     |
|----------|-----------------------------------------------------------------------------------|------------|---------------------------------------------------------------------|
| LOOP（行为） | <b>先执行循环体</b> ，至 LOOP 指令处再做 $CX--$ 与判断；适合已至少执行一次的循环（类似 do-while）。                 | REP 系列（区别） | REP 是“先判后执”：先检查 CX 是否为 0，若为 0 则 <b>不执行</b> 字符串指令。与 LOOP 的先减后判本质不同。  |
| 边界情况     | 若初始 $CX=0$ ，则执行到第一次 LOOP 时 $CX$ 变为 FFFFH（ $-1$ ），非零， <b>故会跳转——循环将执行 65536 次</b> 。 | 等价性注意      | 逻辑上 LOOP 等价于 DEC CX；JNZ label，但不同点在于 DEC 会影响标志位（如 ZF），而 LOOP 不改变标志。 |

|                              |                                                                                                                                                                   |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| LOOPE/LOOPZ<br>LOOPNE/LOOPNZ | 执行： $CX \leftarrow CX - 1$ ；若 $CX=0$ 或 $ZF=0$ 则退出（即“ <b>要么计数耗尽，要么找到了不相等</b> ”）。<br>执行： $CX \leftarrow CX - 1$ ；若 $CX=0$ 或 $ZF=1$ 则跳转（即“ <b>要么计数耗尽，要么找到了相等</b> ”）。 |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|

| 调用返回指令 CALL、RET、RETF |                                                                                      |
|----------------------|--------------------------------------------------------------------------------------|
| <b>● 概述：</b>         | 子程序调用的基本框架。调用/返回通过修改 IP/EIP（必要时并修改 CS）改变执行流                                          |
| CALL（调用）<br>CALL（返回） | 跳转到子程序入口，同时 <b>保存返回地址</b> （即调用指令的下一条指令地址，通常压入栈）。<br>从栈中取出返回地址并跳回，继续执行 CALL 之后的下一条指令。 |

- 注：** RET 始终返回到 CALL 指令的**下一条指令处**；调用/返回可为段内（只修改 IP）或段间（同时修改 CS 与 IP）。

| 段内调用（Intra-segment）                                              | 段间调用（Inter-segment）                                          |
|------------------------------------------------------------------|--------------------------------------------------------------|
| <b>段内直接</b>                                                      | <b>段间直接</b>                                                  |
| 目标在当前代码段内。目标地址直接编码在指令中（如 CALL NEAR label）；仅修改 IP。                | 目标段：偏移作为立即数编码在指令中（如 CALL FAR ptr）；执行时同时修改 CS 与 IP。           |
| <b>段内间接</b>                                                      | <b>段间间接</b>                                                  |
| 目标由寄存器或存储器提供（如 CALL reg / CALL [mem]），取出的 16/32 位偏移赋给 IP（CS 不变）。 | 目标地址存于内存或寄存器（在 8086 为 4 字节：低位偏移先存，随后为段），执行时先读出偏移与段并赋给 IP/CS。 |

|                             |                                                           |
|-----------------------------|-----------------------------------------------------------|
| <b>● 段内直接调用（Near Call）：</b> |                                                           |
| <b>● CALL 的微操作：</b>         |                                                           |
| 压栈                          | 将当前的 IP/EIP（即下一条指令）压入堆栈。<br>栈指针变化： $SP \leftarrow SP - 2$ |
| 跳转                          | 将 IP/EIP 设为目标过程的偏移地址，完成控制流转移。                             |

- 执行流程实例（主程序调用子程序）：**

|             |                                                                                                                            |           |                                                                                             |
|-------------|----------------------------------------------------------------------------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------|
| SS          | <div> <div> <div>SP</div> <div>已入栈的数据</div> </div> <div> <div>栈顶（低地址）</div> <div>……</div> <div>栈底（高地址）</div> </div> </div> | 堆栈        | 堆栈结构（先进后出）                                                                                  |
| 初始状态        | 主程序地址：1000H:0100H；指令：CALL NEAR PTR DELAY；指令长度 = 3 字节，故下一条指令地址为 0103H。                                                      | CALL 执行瞬间 | 将 0103H 压入堆栈（ $SP \leftarrow SP - 2$ ），然后 $IP \leftarrow 0200H$ （子程序入口）。                    |
| 子程序内部（保护现场） | 例如执行 PUSH CX 保存寄存器，业务逻辑结束前用 POP CX 恢复。 <b>进入子程序时压入了多少数据，返回前必须弹出相同数量，保证堆栈平衡</b> ，否则 RET 会把错误的值当作返回地址弹出，导致异常。                | 返回（示例）    | 子程序执行至 RET（示例中为 0207H 处），执行出栈：弹出 0103H 进入 IP， $SP \leftarrow SP + 2$ ，程序继续在 1000H:0103H 执行。 |

- 提示：** 栈顶（Top）为低地址，栈向低地址生长（SP 减小）。
- 段间调用（Far Call）**
- 概述：** 段间调用用于跳转到**另一个代码段**中的子程序；与段内调用不同，需同时保存/恢复 CS 与 IP。主要就是堆栈操作的不同：

| 调用过程（CALL）                                                                                                                                                                                                                   | 返回过程（RET/RETF）                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>将当前 CS 压入堆栈（<math>SP \leftarrow SP - 2</math>）。</li> <li>将当前 IP 压入堆栈（<math>SP \leftarrow SP - 2</math>）；栈顶（SP）处存放 IP，<math>SP + 2</math> 处存放 CS。</li> <li>更新 CS:IP 为目标地址并跳转。</li></ul> | <ul style="list-style-type: none"><li>弹出栈顶字送入 IP，<math>SP \leftarrow SP + 2</math>。</li> <li>继续弹出字送入 CS，<math>SP \leftarrow SP + 2</math>。</li></ul> |

| 中断指令 INT / IRET           |                                                                                                         |
|---------------------------|---------------------------------------------------------------------------------------------------------|
| <b>● 概述：</b>              | 中断是一种“被动”的系统级调用机制，用于在运行时转入中断服务程序（ISR）以处理外部/内部事件。执行 INT n 会 <b>自动保存现场并跳转到向量表指定的处理程序</b> ；返回须用 IRET 恢复现场。 |
| 中断向量表（IVT）<br>向量结构        | 系统内存中共有 <b>256</b> 个向量，每个向量占 <b>4 字节</b> ，共 1KB，向量地址为 $n \times 4$ 。<br>低 2 字节：偏移地址（IP）；高 2 字节：段地址（CS）。 |
| <b>● INT n 的执行步骤：</b>     |                                                                                                         |
| 步骤 1 - PUSHF              | 将标志寄存器 FLAGS 压入堆栈（保存状态）。                                                                                |
| 步骤 2 - 清标志                | 自动将 IF=0、TF=0（屏蔽可屏蔽中断并禁止单步），防止中断嵌套/单步干扰。                                                                |
| 步骤 3 - PUSH CS            | 将当前代码段 CS 压入堆栈。                                                                                         |
| 步骤 4 - PUSH IP            | 将当前指令指针 IP（返回地址）压入堆栈。                                                                                   |
| 步骤 5 - 查表跳转               | 以 $n \times 4$ 为向量地址，取出新的 IP（低字）和 CS（高字），装入寄存器并跳转。                                                      |
| <b>● IRET 一中断返回（微操作）：</b> | 中断服务程序必须用 IRET 返回，以恢复 IP/CS/FLAGS，保持栈平衡与状态一致。                                                           |

|          |                                                       |
|----------|-------------------------------------------------------|
| 微操作（顺序）  | POP IP； POP CS； POP FLAGS（每次 $SP \leftarrow SP + 2$ ） |
| 控制转移指令总结 |                                                       |

|                   |                                                                   |
|-------------------|-------------------------------------------------------------------|
| <b>● 短/长转移限制：</b> | <b>● 核心指令堆栈行为对比：</b>                                              |
| 短程限制              | JMP 不压栈；直接转移（是“干净”）。                                              |
| 无限制               | CALL 压入返回地址：段内压 IP，段间压 CS+IP。<br>INT 压入 FLAGS，CS，IP（现场保护），随后查表跳转。 |

|                  |                                 |
|------------------|---------------------------------|
| <b>● 返回指令对比：</b> |                                 |
| RET              | 与 CALL 配对：弹出 IP（近返）或 IP/CS（远返）。 |
| IRET             | 与 INT 配对：弹出 IP，CS，FLAGS（三字恢复）。  |

|          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 处理器控制类指令 | <b>● 标志操作指令：</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|          | <div> <div>控制标志</div> <div>状态标志</div> </div> <div> <div>15</div><div>14</div><div>13</div><div>12</div><div>11</div><div>10</div><div>9</div><div>8</div><div>7</div><div>6</div><div>5</div><div>4</div><div>3</div><div>2</div><div>1</div><div>0</div> </div> <div> <div>×</div><div>×</div><div>×</div><div>×</div><div>O</div><div>D</div><div>I</div><div>T</div><div>S</div><div>Z</div><div>×</div><div>A</div><div>×</div><div>P</div><div>×</div><div>C</div> </div> <div> <div>溢出标志位</div><div>↑</div> <div>增量方向标志位</div><div>↑</div> <div>允许中断标志位</div><div>↑</div> <div>单步标志位</div><div>↑</div> <div>符号标志位</div><div>↑</div> <div>进位标志位</div><div>↑</div> <div>奇偶标志位</div><div>↑</div> <div>辅助进位标志</div><div>↑</div> <div>零标志位</div><div>↑</div> </div> |

|     |                               |
|-----|-------------------------------|
| CLC | 清除进位标志， <b>CF=0</b> 。         |
| STC | 设置进位标志， <b>CF=1</b> 。         |
| CMC | 进位标志取反， <b>CF=CF</b> 。        |
| CLD | 清除方向标志， <b>DF=0</b> （地址递增）。   |
| STD | 设置方向标志， <b>DF=1</b> （地址递减）。   |
| CLI | 清除中断允许标志， <b>IF=0</b> （禁止中断）。 |
| STI | 设置中断允许标志， <b>IF=1</b> （允许中断）。 |

第三章知识点总结完毕（好多东西啊）

#### 第四章汇编语言程序设计基础

| 概述及语句的组成                         |                                                                    |
|----------------------------------|--------------------------------------------------------------------|
| <b>语言演变</b>                      | 三种语言的映射关系：                                                         |
| <b>● 机器语言（Machine Language）：</b> |                                                                    |
| 计算机硬件唯一能直接识别并执行的语言，由 0/1 二进制码构成。 |                                                                    |
| <b>组成</b>                        | 由机器指令序列构成，每条指令对应 CPU 的基本操作（如取数、加法、跳转）。                             |
| <b>执行机制</b>                      | 无需翻译，CPU 直接读取二进制流并执行。                                              |
| <b>● 汇编语言（Assembly）：</b>         |                                                                    |
| <b>本质属性</b>                      | 本质上是机器语言的符号化，汇编指令与机器指令通常存在一一对应关系。                                  |
| <b>符号化表示</b>                     | 使用 <b>助记符</b> （如 MOV, ADD）与 <b>符号/标号</b> （Labels）代替操作码与地址，便于阅读与编写。 |
| <b>执行流程</b>                      | 源程序 → Assembler（汇编器）→ Linker（链接器）→ 目标机器码 → 执行。                     |

| 汇编程序结构     | ● 指令性语句 vs 指示性语句：                              |
|------------|------------------------------------------------|
| 指令性语句      | 如 MOV/LEA/INT：会被汇编为机器指令执行。                     |
| 指示性语句（伪指令） | 如 SEGMENT/ENDS/ASSUME/DB：用于组织与描述程序，不直接生成可执行指令。 |

|                 |                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                           |
|-----------------|------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 数据段             | <div><div>DATA</div><div>SEGMENT</div><div>STRING DB "HELLO WORLD",0DH,0AH,' ' ; 0DH为换行，0AH为换行</div><div>DATA</div><div>ENDS</div></div> | <div><div>CODE</div><div>SEGMENT</div><div>ASSUME CS:CODE,DS:DATA</div><div>BEGIN</div><div>MOV AX,DATA ; DATA为数据的段地址</div><div>DS AX ; 初始DS寄存器段地址</div><div>MOV AH,09H</div><div>LEA DX,STRING ; 初始字符串</div><div>INT 21H ; AH为09H，INT 21H是DOS.0X号程序(P130)</div><div>MOV AH,4CH</div><div>INT 21H ; AH为4CH时，系统返回DOS(P14)</div><div>CODE</div><div>ENDS</div><div>BEGIN</div></div> <div>代码段（由CPU执行）</div> |
| ● 段定义结构：数据段：    |                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                           |
| DATA<br>SEGMENT | 定义全局变量/常量。                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                           |
| DB              | Define Byte: 定义字节数据。                                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                           |
| ODH, 0AH        | ASCII 的回车 CR 与换行 LF，用于控制输出格式。                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                           |
| \$              | INT 21H 的 09H 功能以 \$ 作为字符串结束标志（必掌握）。                                                                                                     |                                                                                                                                                                                                                                                                                                                                                                                                           |
| ● 段定义结构：代码段：    |                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                           |
| CODE<br>SEGMENT | 存放指令与控制流程。                                                                                                                               |                                                                                                                                                                                                                                                                                                                                                                                                           |
| ASSUME          | 伪指令：将 <b>逻辑段名与段寄存器关联</b> ，便于汇编器进行地址计算。                                                                                                   |                                                                                                                                                                                                                                                                                                                                                                                                           |

**汇编语言语句组成元素** 太多了，挑重点说。

**表达式与运算符 数值运算符**  
运算符在**汇编阶段**由编译器处理，用于将标号或变量转换成具体的数值（地址）；而指令在**运行时**由 CPU 执行。

|                   |                                      |
|-------------------|--------------------------------------|
| <b>● SEG 运算符：</b> | <b>● OFFSET 运算符：</b>                 |
| 定义                | 获取标号或变量所在的 <b>段基地址</b> 。             |
| 返回值               | 冒号前的 16 位段地址，如标号在代码段则为 CS，在数据段则为 DS。 |
| 定义                | 获取标号或变量在该段内的 <b>偏移地址</b> 。           |
| 返回值               | 冒号后的 16 位偏移量，距离段起始位置的字节数。            |

- 实例分析：** 假设标号 ADDR：INC CX 的逻辑地址为 2000H:1234H。

|                  |                          |                     |                          |
|------------------|--------------------------|---------------------|--------------------------|
| MOV AX, SEG ADDR | 提取段基地址。<br>结果：AX = 2000H | MOV AX, OFFSET ADDR | 提取偏移地址。<br>结果：AX = 1234H |
|------------------|--------------------------|---------------------|--------------------------|

- 关键性质：**

等价指令 MOV AX, OFFSET ADDR 等价于 LEA AX, ADDR：两者均得到标号的偏移地址。  
汇编时计算 OFFSET 在汇编阶段确定，机器码直接包含立即数；LEA 为运行时计算的指令，但结果相同。

| 汇编语言的语句           |                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------|
| <b>指示性语句</b>      |                                                                                           |
| <b>指示性语句（伪指令）</b> | <b>● 定义：</b> 指示性语句（伪指令）不被翻译成机器指令，CPU 无法直接执行。 <b>这些语句是写给汇编器的</b> ，用于指导汇编器分配内存、定义段、指定程序入口等。 |
| <b>常用伪指令</b>      |                                                                                           |

|                |                                                                                                                                                                                        |
|----------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| SEGMENT / ENDS | 定义一个逻辑段的起始和结束。<br><b>格式：</b> <名字 > SEGMENT ... <名字 > ENDS<br>数据定义伪指令。<br>DB：定义字节数据；DW：定义字（2 字节）；DD：定义双字（4 字节）。<br>例：MSG DB 'Hello', 0DH, 0AH, '\$'（定义字符串，末尾需 \$ 作为 INT 21H 09H 的结束标志）。 |
| DB / DW / DD   |                                                                                                                                                                                        |
| DUP            | 重复操作符，用于快速分配未初始化或重复的存储空间。<br>例：BUFFER DB 256 DUP (?)（分配 256 字节未初始化区域）；ZERO DB 100 DUP (0)（分配 100 个字节全为 0）。                                                                             |
| ASSUME         | 将逻辑段名与段寄存器关联，便于汇编器进行地址计算。<br>例：ASSUME CS:CODE, DS:DATA, SS:STACK。 <b>重点：仅供汇编器使用，不直接装入寄存器。</b>                                                                                          |
| END            | 标记源程序结束。通常后跟程序入口标号。<br>例：END START（程序从 START 标号处开始执行）。                                                                                                                                 |
| OFFSET         | 返回标号或变量的段内偏移地址（16 位）。<br>例：MOV AX, OFFSET LABEL（等价于 LEA AX, LABEL）。                                                                                                                    |
| SEG            | 返回标号或变量所在的段基址（16 位段地址）。<br>例：MOV AX, SEG DATA（取数据段的段基址）。                                                                                                                               |

- 标号 vs 名字：** 标号（如 LOOP:）通常后跟冒号，代表指令地址，用于 JMP/CALL 跳转。名字（如段名、变量名）不加冒号，是伪指令的标识符。
- 指令性语句** 好像都是 DOS 系统里的一些调用，暂时不看了。

| 完整的汇编程序         |                                                                |
|-----------------|----------------------------------------------------------------|
| <b>汇编程序开发流程</b> | 一共分为四个阶段：                                                      |
| 编辑（Edit）        | 用文本编辑器编写源代码。产物：.asm（源程序文件）。                                    |
| 汇编（Assemble）    | 用汇编器（如 MASM）将 .asm 翻译为目标文件。产物：.obj（目标文件，符号/地址尚未定）。             |
| 连接（Link）        | 用链接器（如 LINK）将一个或多个 .obj 与库文件合并并进行地址定位。产物： <b>.exe（可执行文件）</b> 。 |
| 调试（Debug）       | 用调试器（如 DEBUG）在 .exe 上单步执行、查看寄存器/内存，定位并修正缺陷。                    |
| <b>● 要点：</b>    |                                                                |
| 不可直接运行          | .obj 不能直接执行，需链接生成 .exe。                                        |
| 职责划分            | 汇编器负责机器码生成， <b>链接器负责符号解析与地址重定位</b> ，调试器负责运行时检查。                |
| <b>程序分段设计</b>   | <b>数据段与堆栈段</b>                                                 |

|            |                 |                            |             |
|------------|-----------------|----------------------------|-------------|
| <b>数据段</b> | DATA            | SEGMENT                    | ； 数据段       |
|            | .....           |                            | ； 定义变量、缓冲区等 |
|            | DATA            | ENDS                       |             |
| <b>堆栈段</b> | STACK           | SEGMENT PARA STACK 'STACK' | ； 堆栈段部分     |
|            | DB XXXX DUP (?) |                            | ； 定义堆栈的长度   |
|            | STACK           | ENDS                       |             |





|        |                                                                                                                                                                                                                             |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 基本性能指标 | <p>存取时间 从 CPU 发出地址信号到数据有效（读）或写入完毕（写）的时间。</p> <p>存储周期 进行一次完整读写所需的最小时间间隔。</p> <p>二者关系 <b>存储周期 &gt; 存取时间</b>（内部电路需恢复时间）。</p> <p>读/写周期 <math>t_{\text{cyc}}(R)</math>: 连续两次读最小间隔；<math>t_{\text{cyc}}(W)</math>: 连续两次写最小间隔。</p> |
|--------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

计算末尾地址的公式为：
末尾地址 = 首地址 + 存储容量 - 1
注：减 1 是因为地址是从 0 开始计数的，且首地址本身占了一个存储单元。

|     |                               |                                                                                            |
|-----|-------------------------------|--------------------------------------------------------------------------------------------|
| 地址线 | $A_0 \sim A_n$                | 接地址总线 $AB$ ，输入信号，决定芯片容量 $N = 2^{n+1}$ 。                                                    |
| 数据线 | $D_0 \sim D_m$                | 接数据总线 $DB$ ，双向（RAM）或输出（ROM），决定字长 $M$ 。                                                     |
| 片选线 | $\overline{CS}/\overline{CE}$ | 由 <b>高位地址译码</b> 产生，低电平有效，用于选中该芯片。                                                          |
| 读写线 |                               | 读允许 $\overline{OE}$ 接 CPU 的 $\overline{RD}$ ；写允许 $\overline{WE}$ 接 CPU 的 $\overline{WR}$ 。 |

|         |                                                                                                    |
|---------|----------------------------------------------------------------------------------------------------|
| 6264 组织 | 地址线 $A_0 \sim A_{12}$ ( $2^{13} = 8K$ )；数据线 $D_0 \sim D_7$ 。                                       |
| 双片选机制   | $\overline{CS1}$ （低有效）和 $\overline{CS2}$ （高有效）。选中条件： $\overline{CS1} = 0$ 且 $\overline{CS2} = 1$ 。 |
| 读写控制    | $\overline{OE}$ （输出允许，接系统 $\overline{RD}$ ）； $\overline{WE}$ （写允许，接系统 $\overline{WR}$ ）。           |

|      |       |                                                                             |
|------|-------|-----------------------------------------------------------------------------|
| 2114 | 容量与组织 | $1K \times 4$ 位 (0.5 KB)；地址线 $A_0 \sim A_9$ ( $2^{10}$ )，数据线 $D_1 \sim D_4$ |
|      | 位扩展   | 8 位系统需 <b>两片并联</b> ，分别负责高 4 位和低 4 位                                         |

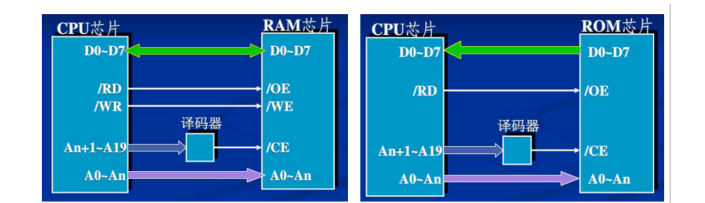
由于只有一个  $\overline{WE}$  而没有  $\overline{OE}$ ，因此 2114 的读写模式由  $\overline{CS}$  和  $\overline{WE}$  共同决定：

|      |                                                       |
|------|-------------------------------------------------------|
| 写模式  | $\overline{CS} = 0, \overline{WE} = 0$ ，数据端口为输入（DIN）  |
| 读模式  | $\overline{CS} = 0, \overline{WE} = 1$ ，数据端口为输出（DOUT） |
| 待机模式 | $\overline{CS} = 1$ ，输出为 <b>高阻态（Hi-Z）</b> ，实现总线隔离     |

|       |                                                         |
|-------|---------------------------------------------------------|
| 容量与组织 | 地址线 $A_0 \sim A_{12}$ ；数据线 $D_0 \sim D_7$ 。与 6264 引脚兼容。 |
| CE#   | 片选使能，低电平有效。控制芯片激活及低功耗模式。                                |
| OE#   | 输出使能，低电平有效。读操作时打开输出缓冲器。                                 |
| PGM#  | 编程脉冲， <b>低电平有效</b> 。烧录时施加负脉冲，正常读取时置高。                   |
| Vpp   | 编程电压引脚，烧录时需施加高压（12.5V 或 21V）。                           |

在对芯片进行数据烧录（编程）时，需要在 PGM# 施加特定宽度（如 50ms）的负脉冲，配合  $V_{pp}$  引脚的高压（通常 12.5V 或 21V），将数据写入**浮栅晶体管**（EPROM 没有  $\overline{WE}$  引脚，因为它在正常工作时不可写。）

低位地址线用于片内寻址，高位地址线用于片选寻址：

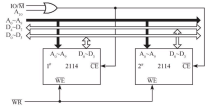


**存储器扩展**

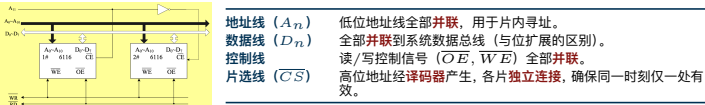
- 存储器扩展**：当单片芯片容量（字数  $N$  或字长  $M$ ）不足以满足系统需求时，需通过多片组合进行扩展。

|        |                                                                                                                                           |
|--------|-------------------------------------------------------------------------------------------------------------------------------------------|
| 位扩展    | 增加 <b>字长</b> ，字数不变。 $N \times M \rightarrow N \times (M \times k)$ 。各片 <b>地址线、读写线并联</b> ，数据线分别连接 CPU 数据总线的不同位。                            |
| 字扩展    | 增加 <b>字数</b> （容量），字长不变。 $N \times M \rightarrow (N \times k) \times M$ 。各片 <b>数据线、低位地址线并联</b> ，高位地址线经 <b>译码器</b> 产生片选信号 $\overline{CS}$ 。 |
| 位和字节扩展 | 同时增加字长和字数。通常先通过位扩展组成满足字长要求的“存储组”，再通过字扩展（译码片选）连接各组。                                                                                        |

**位扩展** 当芯片字长小于 CPU 数据总线宽度时，通过多片并联增加**存储字长**。如两片  $1K \times 4$  的 2114 并联可组成  $1K \times 8$  存储器。



**字节扩展** 当芯片字长满足但容量不足时，增加**存储单元总数**（地址空间深度）。



**地址线** ( $A_n$ ) 低位地址线**全部并联**，用于片内寻址。

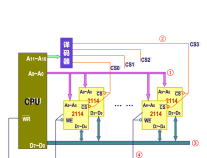
**数据线** ( $D_n$ ) **全部并联**到系统数据总线（与位扩展的区别）。

**控制线** 读/写控制信号 ( $\overline{OE}$ ,  $\overline{WE}$ ) **全部并联**。

**片选线** ( $\overline{CS}$ ) 高位地址经**译码器**产生，各片**独立连接**，确保同一时刻仅一处有效。

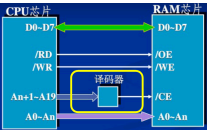
● **核心逻辑**：通过高位地址译码，将不同芯片映射到系统内存空间的不同地址段（如 0000H ~ 1FFFH）。

**位和字节扩展** 当芯片字长和容量均不足时，需同时进行扩展。



**地址译码电路设计（片选）**

- 译码电路设计**：
  - 片内寻址**：根据芯片容量  $N$  确定低位线数  $k$  ( $2^k = N$ )，如 4KB 需  $A_{11} \sim A_0$ 。
  - 片选逻辑**：高位地址通过译码器（如 74LS138）或逻辑门产生  $\overline{CS}$  #。
  - 范围确定**：首地址为高位固定、低位全 0；末地址为高位固定、低位全 1。



**地址重叠计算**：若有  $n$  根高位地址线未参与译码，则一个物理单元对应  $2^n$  个重叠地址。

**全地址译码** 这个后面的基本上都是以画图计算为主，就直接上题了：

**习题解析**

- 作业 4-1**：半导体存储器分几个类型？各自的特点是什么？

● **解**：从存取方式来分，有读写存储器 RAM 和只读存储器 ROM。RAM 也称为随机存取存储器，CPU 在执行程序的过程中能对它进行读出和写入操作。在 RAM 中可分为双极型（Bipolar）和 MOS 型 RAM 两大类。双极型 RAM 具有很高的存取速度，但是双极型 RAM 的集成度低，单片容量小，功耗大，成本高。MOS 型 RAM 具有功耗低、集成度高、单片容量大的特点，但存取速度则较慢。MOS 型 RAM 又可以分为静态 RAM 和动态 RAM 两种。ROM 器件的功能是只许读出，不许写入，一旦有了信息，就不能轻易改变，也不会存在掉电时丢失，所以它只能用在不需要经常对信息进行修改和写入的地方

● **PS**：这个标准答案真的没边了

● **作业 4-2**：试比较 SRAM 和 DRAM 的优缺点。

● **解**：SRAM 的优点是存取速度快，不需要刷新，结构简单，控制电路简单，易于与 CPU 直接相连；缺点是集成度低，单片容量小，功耗大，成本高。DRAM 的优点是集成度高，单片容量大，功耗低，成本低；缺点是存取速度慢，需要定期刷新，控制电路复杂，不易与 CPU 直接相连。

● **作业 4-3**：若某微机系统的系统 RAM 存储组由 4 个模块组成，每个模块的容量为 128KB，若 4 个模块的地址是连续的，最低地址为 00000H，试指出每个模块的首末地址。

● **解**：每个模块容量为 128KB =  $2^{17}$  B，需 17 根地址线。模块地址连续，地址范围如下：

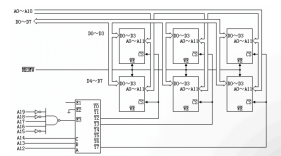
- 模块 0：首地址 00000H，末地址 1FFFFH；
  - 模块 1：首地址 20000H，末地址 3FFFFH；
  - 模块 2：首地址 40000H，末地址 5FFFFH；
  - 模块 3：首地址 60000H，末地址 7FFFFH。
- **作业 4-4**：对于下列芯片，它们的片内地址线各有多少条？若分别用以下芯片组成容量为 64KB 的模块，试指出分别需要多少芯片。
- Intel 2114 ( $1K \times 4$  位)；
  - Intel 6116 ( $2K \times 8$  位)；
  - Intel 2164 ( $64K \times 1$  位)；
  - Intel 3148 ( $4K \times 8$  位)。

● **解**：

- $1K = 2^{10}$ ，地址线  $A_0 \sim A_9$  共 10 条。每片容量  $1K \times 4 = 512B$ ，需  $64KB/512B = 128$  片。
- $2K = 2^{11}$ ，地址线  $A_0 \sim A_{10}$  共 11 条。每片容量  $2K \times 8 = 2KB$ ，需  $64KB/2KB = 32$  片。
- $64K = 2^{16}$ ，地址线  $A_0 \sim A_{15}$  共 16 条。每片容量  $64K \times 1 = 8KB$ ，需  $64KB/8KB = 8$  片。
- $4K = 2^{12}$ ，地址线  $A_0 \sim A_{11}$  共 12 条。每片容量  $4K \times 8 = 4KB$ ，需  $64KB/4KB = 16$  片。

● **作业 4-5**：某 RAM 芯片，容量为  $4K \times 4$  位，该芯片有数据线、地址线、片选信号线  $\overline{CS}$  和读写控制线  $\overline{WR}$ 。请问：

- 该 RAM 芯片有几条地址线？几条数据线？
- 主要要在以 8088 为 CPU 的微机系统中，用该芯片构成图 6-35 中所示的 RAM#2 和 RAM#4 两个内存模块，请画出扩展这两个模块的存储器连接图。（连接图中可选用 3:8 译码器和与非门等。）



**存储器与 CPU 连接**

## 第七章 I/O 接口

**概述** CPU 对内存和外设的**操作本质上**是**读与写**。接口电路在 CPU 与外设之间实现信息交换，主要功能包括：

|      |                                                               |
|------|---------------------------------------------------------------|
| 地址选择 | 选择输入或输出的外设设备                                                  |
| 控制功能 | 启动、停止或复位外设                                                    |
| 时序匹配 | 通过状态信号解决时序不匹配                                                 |
| 速度匹配 | 通过缓冲或握手信号解决速度不匹配                                              |
| 格式转换 | 将信息格式调整为 CPU 和外设均可接受的形式                                       |
| 电平转换 | 将 <b>TTL/CMOS 电平</b> (0V-5V) 与其他标准（如 RS-232 的 $\pm 12V$ ）相互转换 |
| 可编程性 | 通过控制寄存器提高硬件通用性                                                |

● **输入/输出指令**：

|              |                              |
|--------------|------------------------------|
| IN DST, SRC  | 从外设端口读取数据到 CPU 寄存器（外设 → CPU） |
| OUT DST, SRC | 将 CPU 数据输出到外设端口（CPU → 外设）    |

**I/O 端口的编址方式**

- I/O 端口与接口概念：

|        |                                               |
|--------|-----------------------------------------------|
| I/O 接口 | 物理芯片或集成电路，是 CPU 与外设的中介。                       |
| I/O 端口 | <b>接口内部的寄存器（数据/状态/控制）</b> ，是逻辑单元。每个端口对应固定的地址。 |

● **编址关系**：一个 I/O 接口若含有  $n$  个端口，则占用  $n$  个独立地址。

**编址方式分类**

|                  |                                                                                                                                                                                                                                        |
|------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 独立编址 (8086/8088) | CPU 开辟两个独立的地址空间：内存地址空间与 I/O 地址空间。                                                                                                                                                                                                      |
| 统一编址 (C51 单片机)   | <p><b>区分机制</b>：专用控制信号（如 <math>M/\overline{TO}</math>）或专用指令（IN/OUT）。</p> <p><b>优势</b>：内存与 I/O 互不干扰，便于管理；I/O 地址空间通常较小（64KB）。I/O 端口被视为内存的一部分，统一编址。</p> <p><b>访问方式</b>：使用通用访存指令（如 MOV），无需专用 I/O 指令。</p> <p><b>特点</b>：编址简单但占用内存空间；寻址灵活。</p> |

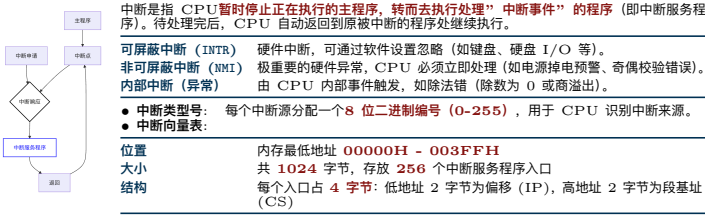
|                                                                             |                                                                 |
|-----------------------------------------------------------------------------|-----------------------------------------------------------------|
| 独立编址                                                                        | ● <b>概述</b> ：                                                   |
| 特点                                                                          | CPU 通过 <b>专用控制信号</b> （如 $M/\overline{TO}$ ）区分访问目标为存储器还是 I/O 端口。 |
| 地址空间                                                                        | I/O 地址与内存地址 <b>互相独立</b> ，互不占用。                                  |
| 指令系统                                                                        | 需设置 <b>专用 I/O 指令</b> （IN/OUT）进行 I/O 访问。                         |
| 总线信号区分                                                                      | <b>8086 I/O 地址空间</b>                                            |
| $M/\overline{TO} =$                                                         | 内存访问 1                                                          |
| $M/\overline{IO} =$                                                         | I/O 访问 0                                                        |
| 通过此信号与 $\overline{RD}/\overline{WR}$ 组合，CPU 产生 4 种总线周期：内存读、内存写、I/O 读、I/O 写。 |                                                                 |
| ● <b>优缺点对比</b> ：                                                            |                                                                 |

|      |                       |                                     |
|------|-----------------------|-------------------------------------|
| 优点   |                       | <b>缺点</b>                           |
| 空间独立 | I/O 不占用内存地址，存储器容量最大化。 | <b>指令复杂</b> 需专用 I/O 指令，CPU 设计复杂度增加。 |
| 总线效率 | I/O 访问地址位数可较少，提高利用率。  | <b>硬件复杂</b> 需独立的 I/O 控制逻辑与地址译码电路。   |

|       |                                    |                                            |
|-------|------------------------------------|--------------------------------------------|
| 统一编址  |                                    | ● 概述: 将 I/O 端口映射到内存地址空间的一部分, 使用访存指令操作 I/O。 |
| 优点    |                                    | 缺点                                         |
| 指令通用性 | 使用所有访存指令 (MOV, ADD, AND 等) 操作 I/O。 | 地址空间占 I/O 端口挤占内存寻址空间, 可用存储器减少。             |
| 硬件简化  | 无需专用控制线, 仅使用内存读写信号。                | 传输开销增 总线传送地址位数相同, 可能增加系统负载。                |

## I/O 接口的基本工作方式

**中断** 比较重要的概念，这里单拿出来讲



● **中断断点**：

|    |                                            |
|----|--------------------------------------------|
| 定义 | <b>中断发生时即将执行但尚未执行的指令地址</b> （当前指令的下一条）      |
| 处理 | CPU 自动将断点的 <b>CS:IP</b> 压入堆栈保存，以便中断返回后继续执行 |

|        |                              |
|--------|------------------------------|
| 中断响应过程 | <b>响应条件</b>                  |
| 内部中断   | 等到当前指令执行结束                   |
| NMI    | 总线控制权在 CPU，等到当前指令结束          |
| INTR   | IF = 1（中断允许），无更高优先级请求，当前指令结束 |

|             |                                  |
|-------------|----------------------------------|
| CPU 中断响应微操作 |                                  |
| 1. 保存现场     | 将 FLAGS、CS、IP 依次压入堆栈             |
| 2. 清除 IF    | 禁止响应其他可屏蔽中断，防止混乱                 |
| 3. 清除 TF    | 避免单步跟踪 ISR                       |
| 4. 查表跳转     | 根据中断类型号查向量表获得中断服务程序入口 CS:IP，跳转执行 |
| 程序结构        |                                  |
| 保护现场        | 手动 PUSH 需使用的寄存器（AX、BX 等）         |
| 开中断（STI）    | 将 IF 置 1，允许中断嵌套                  |
| 执行主体        | 处理中断事件的具体代码                      |
| 关中断（CLI）    | 将 IF 置 0，保护恢复现场的原子性              |
| 恢复现场        | 按压入顺序反向 POP 出所有寄存器               |
| 返回（IRET）    | 弹出 IP、CS、FLAGS，返回主程序             |

**中断的定义与优先级**

- 定义**： CPU 暂停正在执行的程序，转而执行处理中断事件的程序（中断服务程序），完成后自动返回继续原程序。

|           |                                 |
|-----------|---------------------------------|
| 内部中断      | CPU 内部事件触发（如除法规、溢出）。            |
| NMI（非可屏蔽） | 极重要的硬件异常，必须立即处理（如电源掉电、奇偶校验错）。   |
| INTR（可屏蔽） | 一般硬件中断，可通过 IF 标志屏蔽（如键盘、硬盘 I/O）。 |

● **中断优先级（高 → 低）**：内部中断 → NMI → INTR → 单步中断



| INTR（可屏蔽中断） |                                | NMI（不可屏蔽中断） |                |
|-------------|--------------------------------|-------------|----------------|
| 控制          | 由标志寄存器 IF 控制。IF=1 允许, IF=0 屏蔽。 | 类型号         | 2。             |
| 类型号         | 8~255（由 8259A 提供）。             | 特性          | 不受 IF 控制，必须响应。 |
| 指令          | CLI 禁止, STI 允许。                |             |                |

| 内部中断        |                      |
|-------------|----------------------|
| 除法错 (INT 0) | 除数为 0 或商溢出。          |
| 单步 (INT 1)  | TF=1 时每条指令后触发 (调试用)。 |
| 断点 (INT 3)  | 执行 INT 3 指令。         |
| 溢出 (INT 4)  | 执行 INTO 指令且 OF=1。    |
| 软中断 (INT n) | 执行 INT n 指令。         |

| 中断向量表 |                                                                |
|-------|----------------------------------------------------------------|
| 位置    | 物理地址 00000H~003FFH (1KB)。                                      |
| 结构    | 256 个向量, 每个占 4 字节。向量 n 的地址为 4n, 存放中断服务程序入口 CS:IP。              |
| 布局    | 地址 4n: IP 低字节; 4n + 1: IP 高字节; 4n + 2: CS 低字节; 4n + 3: CS 高字节。 |

**中断响应过程**
● **响应条件**：当前指令执行完毕且满足条件（INTR 则需 IF=1，内部中断无延迟）。

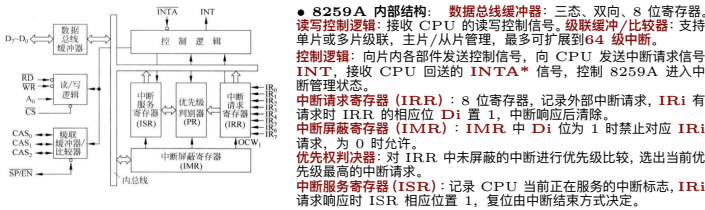
- CPU 微操作**：

|            |                         |
|------------|-------------------------|
| 1. PUSHF   | 将 FLAGS 压栈。             |
| 2. 清 IF/TF | 禁止中断嵌套与单步。              |
| 3. PUSH CS | 将代码段压栈。                 |
| 4. PUSH IP | 将指令指针（断点）压栈。            |
| 5. 查表跳转    | 读取向量表获 CS:IP，跳转至中断服务程序。 |

|                                         |                    |
|-----------------------------------------|--------------------|
| PUSH AX, BX, CX, . . . <span> </span> ； | 保护现场               |
| STI <span> </span> ；                    | 允许嵌套中断（可选）         |
| ● 中断处理逻辑                                |                    |
| CLI <span> </span> ；                    | 禁止中断（恢复现场前）        |
| POP CX, BX, AX, . . . <span> </span> ；  | 恢复现场               |
| IRET <span> </span> ；                   | 返回（弹出 IP/CS/FLAGS） |

- 关键点**：IRET 自动弹出 IP→CS→FLAGS 三次，恢复程序现场；栈平衡至关重要。

**8259A 及其应用**
● **8259A 特性·功能**：8259A 是一个功能很强的中断扩充和多重中断管理芯片，具有**中断扩展、自动提供中断类型码、中断优先级裁决**等中断管理功能。**可编程**：内部有多个寄存器以及功能部件都是可编程的，使用方便。**级联扩展**：单片可连接**8 个中断请求源**，多片级联可扩展到**64 级中断**。通过编程可设置**中断触发方式、中断类型码、中断屏蔽方式、中断优先级方式、中断结束方式**等。



● **8259A 内部结构**：
**数据总线缓冲器**：三态、双向、8 位寄存器。**读写控制逻辑**：接收 CPU 的读写控制信号。**级联缓冲/比较器**：支持单片或多片级联，主片/从片管理，最多可扩展到**64 级中断**。**控制逻辑**：向片内各部件发送控制信号，向 CPU 发送中断请求信号 **INT**，接收 CPU 回送的 **INTA\*** 信号，控制 8259A 进入中断管理状态。

**中断请求寄存器 (IRR)**：8 位寄存器，记录外部中断请求，**IRi** 有请求时 IRR 的相应位 **D<sub>i</sub>** 置 1，中断响应后清除。**中断屏蔽寄存器 (IMR)**：**IMR** 中 **D<sub>i</sub>** 位为 1 时禁止对应 **IRi** 请求，为 0 时允许。

**优先权判选器**：对 IRR 中未屏蔽的中断进行优先级比较，选出当前优先级最高的中断请求。

**中断服务寄存器 (ISR)**：记录 CPU 当前正在服务的中断标志，**IRi** 请求响应时 ISR 相应位置 1，复位由中断结束方式决定。

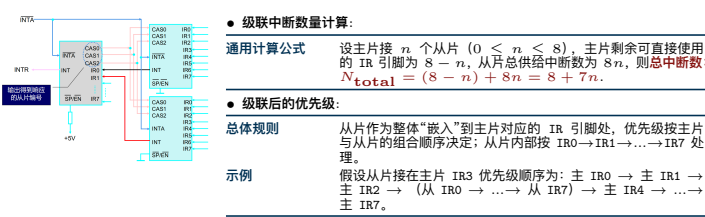
|                 |                                                                                                    |
|-----------------|----------------------------------------------------------------------------------------------------|
| 8259A 发起中断      | 选出有效请求后，8259A 在 INT 引脚向 CPU 的 INTR 发出中断请求信号。                                                       |
| CPU 第一次响应       | CPU 在合适时刻产生第一个 INTA 负脉冲作为握手；该周期数据总线为高阻态， <b>仅表示 CPU 已准备响应。</b>                                     |
| 寄存器更新 (ISR IRR) | 在收到第一个 INTA 后，8259A 将对应中断位置入 ISR（服务中标志）并清除 IRR 中该位                                                 |
| 第二次响应与向量传输      | CPU 发第二个 INTA 负脉冲；8259A 在此期间把中断类型号放到数据总线 D <sub>7</sub> D <sub>0</sub> ，CPU 读取该码以查向量表获得 CS:IP 并跳转。 |

| 第三阶段 — 中断服务与结束        |                                                                           |
|-----------------------|---------------------------------------------------------------------------|
| 执行中断服务                | CPU 使用类型号查表并跳转执行 ISR；8259A 对应的 ISR 位保持为 1，阻止相同或低优先级中断打断（除非允许嵌套）。          |
| EOI(End of Interrupt) | 中断服务结束时软件须向 8259A 发送 EOI（若为非自动 EOI 模式），8259A 清除相应 ISR 位；自动 EOI 模式由硬件完成清位。 |

**8259A 的级联**
主从片区分在于 SP/EN 引脚：主片 SP/EN=1，从片 SP/EN=0，主片的 *CAS*<sub>0</sub> ~ *CAS*<sub>2</sub> 与所有从片的 *CAS*<sub>0</sub> ~ *CAS*<sub>2</sub> 对应相连：主片输出，从片输入。
当从片的中断请求被响应时，主片利用这三根线广播当前被响应的从片 ID 号。此时，主片将对应的从片 ID 发送到 *CAS* 总线上，所有从片都会读取 *CAS* 总线上的 ID，并与自己在初始化时 (ICW3) 设定的 ID 进行比较。**只有 ID 匹配的那个从片，才会把中断类型码投放到数据总线上。**

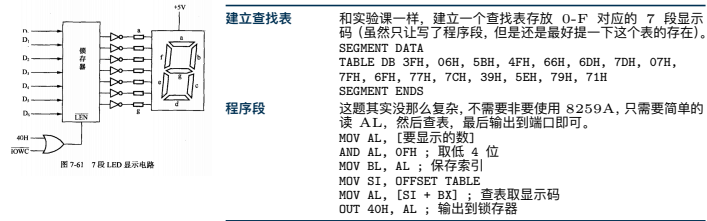
**第 1 个 INTA** CPU 发出第一次握手脉冲，8259A **主/从片冻结当前请求并准备应答**  
**第 2 个 INTA** —主片 若被选中请求来自从片，主片不在数据总线上驱动数据，而通过 *CAS*<sub>0</sub> ~ *CAS*<sub>2</sub> **广播被选从片的 ID**。

**第 2 个 INTA** —从片 与广播 ID 匹配的从片在此时将其中断类型码置于数据总线上供 CPU 读取；其它从片保持高阻态，不干扰总线。



**习题解析** ● 作业 5-1: 用 74LS138 译码器及其他门电路设计一个端口地址译码器, 使 CPU 可以对以下地址范围寻址: 740H 747H; 750H 757H; 758H 75FH; 768H 76FH。

- 解: 当时画的图实在是太丑了, 就不贴了, 讲一下思路: 要确定寻址的地址的统一段, 片选段和片内段, 很容易能找到 3-5 是片选段, 正好接给 74LS138 的 A0-A2; 剩下的使能位就使用剩下能用的高位引脚便能了, 这题按来说应该是上一章的。
- 作业 5-2: 如图所示为 7 段显示器接口, 显示器采用共阳极接法, 试编写程序段, 使 AL 中的一位十六进制数显示于显示器上。输出锁存器地址为 40H。



- 注: 这个可以考前看一眼, 避免对代码过于生疏
- 作业 5-3: 已知 ( $SP$ ) = 0100H, ( $SS$ ) = 0300H, ( $CS$ ) = 0900H, ( $IP$ ) = 00A2H, ( $PSW$ ) = 0240H, 以及 00020H00023H 单元的内容分别是 40H、00H、00H 和 01H, 求执行 INT 8 指令并进入该指令响应的中断断程时,  $SP$ 、 $SS$ 、 $IP$ 、 $CS$ 、 $PSW$  和堆栈最上边 3 个字的内容。

|                              |                             |
|------------------------------|-----------------------------|
| ● 解:                         |                             |
| CPU 执行 INT 8 指令时, 自动进行以下微操作: |                             |
| 步骤 1                         | 将标志寄存器 FLAGS (PSW) 压入堆栈     |
| 步骤 2                         | 将指令指针 IP 压入堆栈               |
| 步骤 3                         | 将代码段寄存器 CS 压入堆栈             |
| 步骤 4                         | 查询中断向量表获得中断服务程序入口, 更新 CS:IP |

- 栈指针变化: 初始  $SP$  = 0100H, 每压入一个字 (16 位) 减 2:

|          |                            |
|----------|----------------------------|
| 压入 PSW 后 | $SP_1$ = 0100H - 2 = 00FEH |
| 压入 CS 后  | $SP_2$ = 00FEH - 2 = 00FCH |
| 压入 IP 后  | $SP_3$ = 00FCH - 2 = 00FAH |

最终  $SP$  = 00FAH。  
● 堆栈内容分析:

|                                 |  |
|---------------------------------|--|
| 最上面第 1 个字 原 IP = 00A2H (00FAH)  |  |
| 最上面第 2 个字 原 CS = 0900H (00FCH)  |  |
| 最上面第 3 个字 原 PSW = 0240H (00FEH) |  |

- 新的寄存器值: 中断向量表位置:  $4 \times 8 = 0020H$ , 存放 INT 8 的中断服务程序入口地址。题目给出 00020H ~ 00023H 内容: 40H、00H、00H、01H。

|      |                   |
|------|-------------------|
| 新 IP | = 0040H (来自向量表低字) |
| 新 CS | = 0100H (来自向量表高字) |

新 PSW: CPU 自动清除 IF 和 TF 位 (禁止中断, 退出单步模式)。

|                        |                                       |
|------------------------|---------------------------------------|
| 原 PSW                  | = 0240H = 0000 0010 0100 0000B        |
| 清除 IF (位 9) 与 TF (位 8) | = 0000 0000 0100 0000B = <b>0040H</b> |

|            |                                                                       |
|------------|-----------------------------------------------------------------------|
| ● 最终答案:    |                                                                       |
| SP         | = 00FAH                                                               |
| SS         | = 0300H (不变)                                                          |
| CS         | = 0100H (新)                                                           |
| IP         | = 0040H (新)                                                           |
| PSW        | = 0040H (新)                                                           |
| 堆栈最上面 3 个字 | <b>00FAH</b> 处: 00A2H   <b>00FCH</b> 处: 0900H   <b>00FEH</b> 处: 0240H |

- 作业 5-4: 8088 系统中只有一片 8259A, 端口地址为 02C0H (偶) 和 02C1H (奇)。要求:

- (1) 中断请求采用电平触发
- (2)  $IR_7$  对应中断类型码 23H
- (3) 采用缓冲器工作方式
- (4) 采用普通 EOI 命令 (非自动)

|         |                                                                                                                         |
|---------|-------------------------------------------------------------------------------------------------------------------------|
| ● 解答:   |                                                                                                                         |
| ICW1 构造 | D7 D6 D5 D4 = 0001 (特征位 ICW1); D3 (LTIM) = 0 (电平触发); D1 (SNGL) = 1 (单片); D0 (IC4) = 1 (需要 ICW4)<br>控制字: 00010011B = 13H |
| ICW2 构造 | $IR_7$ 对应中断类型码 23H, 即基值为 16H (23 = 16 + 7)<br>ICW2 = 16H                                                                |
| ICW3 构造 | 单片工作 (SNGL=1), 无级联, ICW3 不写入                                                                                            |
| ICW4 构造 | D0 (PM) = 1 (8086/8088 模式); D1 (AEOI) = 0 (普通 EOI); D3 (BUF) = 1 (缓冲模式); D2 (M/S) = 1 (作为主片)<br>控制字: 00001101B = 0DH    |

|                                            |  |
|--------------------------------------------|--|
| 汇编程序:                                      |  |
| INTA0 EQU 02C0H ; 偶地址端口 (ICW1, ICW2, ICW3) |  |
| INTA1 EQU 02C1H ; 奇地址端口 (ICW2, ICW4, ICW1) |  |
| MOV DX, INTA0 ; 指向偶地址                      |  |
| MOV AL, 13H ; ICW1 = 13H                   |  |
| OUT DX, AL ; 写入 ICW1                       |  |
| MOV DX, INTA1 ; 指向奇地址                      |  |
| MOV AL, 16H ; ICW2 = 16H (基值)              |  |
| OUT DX, AL ; 写入 ICW2                       |  |

- MOV AL, 0DH ; ICW4 = 0DH
- OUT DX, AL ; 写入 ICW4 (DX 仍为 INTA1)

|        |                                |
|--------|--------------------------------|
| ● 关键点: |                                |
| 写入顺序   | ICW1 (偶) → ICW2 (奇) → ICW4 (奇) |
| 地址区分   | ICW1 写偶地址; ICW2/ICW4 写奇地址      |
| 基值计算   | 中断类型码 23 = 基值 16 + IR 位号 7     |

|                         |                                            |
|-------------------------|--------------------------------------------|
| ● 作业 5-5: 8259A 级联初始化编程 |                                            |
| ● 题目条件:                 |                                            |
| 主 8259A                 | 偶地址 20H, 奇地址 21H; IR0 中断类型码 08H            |
| 从 8259A                 | 偶地址 0A0H, 奇地址 0A1H; 接至主 IR2; IR0 中断类型码 70H |
| 触发方式                    | 边沿触发: 所有中断请求均开启 (IMR 清零)                   |
| EOI 方式                  | 普通 EOI 命令 (非自动)                            |
| 引脚配置                    | SP/EN 用作输入 (级联模式)                          |

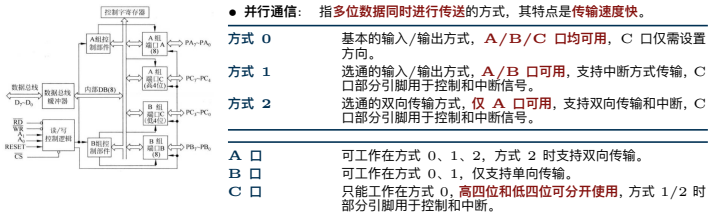
|            |                                                 |
|------------|-------------------------------------------------|
| ● 初始化参数计算: |                                                 |
| 主 8259A    | 从 8259A                                         |
| ICW1       | 00010001B (LTIM=0 边沿; SNGL=0 级联; IC4=1)         |
| ICW2       | 00001000B = 08H (IR0→08H)                       |
| ICW3       | 00000100B = 04H (IR2 有从片)                       |
| ICW4       | 00000001B = 01H (PM=1 为 8086 模式; AEOI=0 普通 EOI) |

|                   |                     |
|-------------------|---------------------|
| ● 汇编程序实现:         |                     |
| MAIN_ICW0 EQU 20H | SLAVE_ICW0 EQU 0A0H |
| MAIN_ICW1 EQU 21H | SLAVE_ICW1 EQU 0A1H |
| MOV AL, 11H       | MOV AL, 11H         |
| MOV DX, MAIN_ICW0 | MOV DX, SLAVE_ICW0  |
| OUT DX, AL        | OUT DX, AL          |
| MOV AL, 08H       | MOV AL, 70H         |
| MOV DX, MAIN_ICW1 | MOV DX, SLAVE_ICW1  |
| OUT DX, AL        | OUT DX, AL          |
| MOV AL, 04H       | MOV AL, 02H         |
| OUT DX, AL        | OUT DX, AL          |
| MOV AL, 01H       | MOV AL, 01H         |
| OUT DX, AL        | OUT DX, AL          |

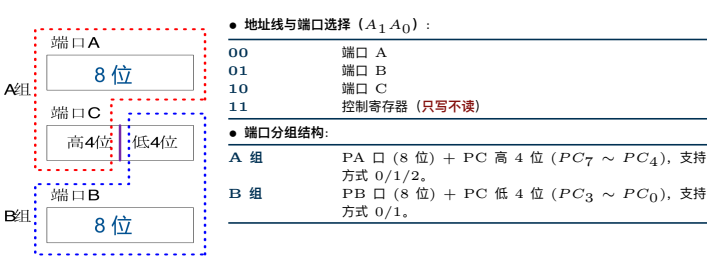
- 核心要点:
- |          |                                                     |
|----------|-----------------------------------------------------|
| 写入顺序     | 主/从片均须遵循 ICW1 (偶) → ICW2 (奇) → ICW3 (奇) → ICW4 (奇)。 |
| 地址区分     | ICW1 写偶地址端口; ICW2/ICW3/ICW4 写奇地址端口。                 |
| 从片 ID 设定 | 从片 ICW3 的低 3 位 (02H) 对应其在主片中的 IR 号 (2)。             |
| 中断向量     | 主 IR0=08H, 从 IR0=70H; 从片 IR1-7 对应 71H-77H。          |

## 第八章常用接口技术

**8255A (可编程并行接口)** ● 8255 功能介绍: 8255 是一种可编程的并行通信接口芯片, 可用于 CPU 和外设之间进行并行数据传输。内部有三个 8 位的数据端口 (24 条 IO 线), 有三种工作方式。端口号的 A0A1 为 00、01、10 分别表示读写 A、B、C 口; 11 表示只写控制寄存器。



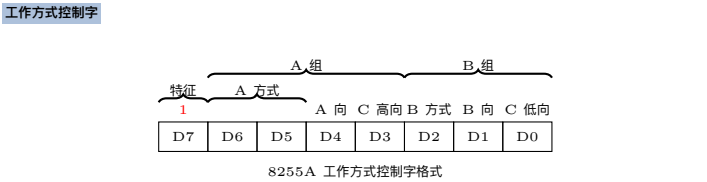
|                                                                           |                                 |
|---------------------------------------------------------------------------|---------------------------------|
| ● 芯片结构                                                                    | ● 硬件结构与端口寻址:                    |
| 数据总线 $D_7 \sim D_0$                                                       | 双向三态 8 位线, CPU 读取状态/写入控制字或传输数据。 |
| 控制信号 $\overline{RD}$ (读有效), $\overline{WR}$ (写有效), $\overline{CS}$ (片选有效) | 均为低电平有效。                        |
| 复位信号 RESET                                                                | 高电平有效, 复位后所有端口置为输入方式。           |
| 三个数据端口                                                                    | PA 口、PB 口、PC 口各 8 位, 与外设连接。     |



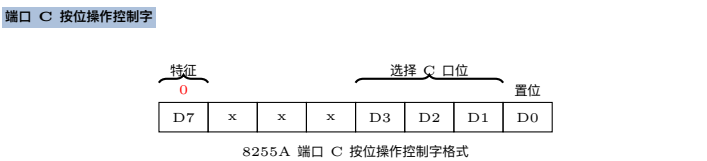
|              |                                    |
|--------------|------------------------------------|
| ● PC 口的双重功能: |                                    |
| 独立 I/O       | 可作为 8 位通用数据端口。                     |
| 握手信号         | 在方式 1/2 中, PC 的部分引脚作为中断请求、应答等控制信号。 |

|               |                                                                                                                           |
|---------------|---------------------------------------------------------------------------------------------------------------------------|
| 工作方式与控制字      | 工作方式                                                                                                                      |
| 方式 0 — 基本 I/O | 适用: A/B/C 口均可用; 特征: 无握手信号, 不使用 PC 控制位; 应用: 同步传输或接入简单外设 (LED、开关等)。                                                         |
| 方式 1 — 选通 I/O | 适用: A/B 口可用; 特征: 带握手信号, PC 的部分位用作选通 ( $\overline{STB}$ )、中断请求 ( $\overline{INTR}$ ) 等控制信号; 应用: 查询/中断方式, 用于速度不匹配或时序不确定的外设。 |
| 方式 2 — 双向传送   | 适用: 仅 A 口可用; 特征: PA 双向传输, 需 5 个 PC 位作握手信号; 应用: 高速双向传输, 查询/中断方式。                                                           |

|         |                                      |
|---------|--------------------------------------|
| 工作方式控制字 | PC 口按位操作控制字                          |
| 功能      | 初始化芯片, 设定 A/B/C 口的工作方向与模式。           |
| 写入时机    | 程序启动时, 一次性配置所有端口。                    |
| 格式      | 8 位数据, 按位编码各口的工作方式 (0/1/2) 与 I/O 方向。 |
| 优势      | 无需读-改-写整个端口, 避免竞态。                   |



|                |                                                                        |
|----------------|------------------------------------------------------------------------|
| ● 工作方式与端口方向设置: | ● 初始化编程示例:                                                             |
| D7 (特征位)       | 必须为 1, 标识该字为方式控制字 (区别于端口 C 按位操作控制字)。                                   |
| D6 - D3 (A 组)  | D6D5 确定工作方式: 00 = 方式 0; 01 = 方式 1; 1X = 方式 2。D4: PA 方向。                |
| D2 - D0 (B 组)  | D2: PC 高 4 位方向。D2: 工作方式 (0 = 方式 0, 1 = 方式 1)。D1: PB 方向。D0: PC 低 4 位方向。 |
| 核心规则           | 1 = 输入; 0 = 输出                                                         |

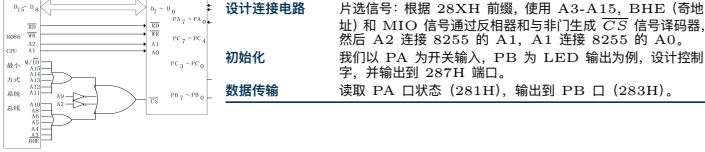


|                                                   |                                                                                                                      |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| D7 (特征位)                                          | 必须为 0, 标识为端口 C 按位操作控制字。                                                                                              |
| D6 - D4                                           | 无关位 (通常置 0)。                                                                                                         |
| D3 - D1 (位选择)                                     | 3 位二进制码选择 $PC_0 \sim PC_7$ 中的某一位: 000 选 $PC_0$ , 111 选 $PC_7$ 。                                                      |
| D0 (操作)                                           | 1: 该位置 1 (高电平); 0: 该位清 0 (低电平)。                                                                                      |
| ● 应用:                                             | 方式 1/2 中 PC 口部分引脚用作握手信号。通过此控制字可在不改变其他位的前提下, 独立控制某一位电平, 避免读-改-写竞态。例如置位 $PC_4$ : MOV AL, 09H (00001001B); OUT 63H, AL。 |
| PC 口按位操作编程实例                                      |                                                                                                                      |
| ● 需求: 8255A 基址为 380H, 要求对 $PC_7$ 置 1, $PC_3$ 清 0。 | ● PC3 清 0:                                                                                                           |
| 控制字构造                                             | $D_7 = 0, D_3 D_2 D_1 = 011, D_0 = 1 \rightarrow 0FH$                                                                |
| 代码                                                | MOV DX, 383H<br>MOV AL, 0FH<br>OUT DX, AL                                                                            |
| ● 要点: 控制字必须写入控制口 (基地址 +3), 即使操作的目标是 PC 口引脚。       |                                                                                                                      |
| 工作方式 0                                            | ● 方式 0 (基本 I/O):                                                                                                     |



|       |                                                                                                                         |
|-------|-------------------------------------------------------------------------------------------------------------------------|
| 特征    | 不需握手信号，CPU 假定外设随时就绪；适用于同步传送或简单外设（如 LED、开关）。                                                                             |
| 独立性   | A 口、B 口、C 高 4 位、C 低 4 位 <b>可独立定义方向</b> （1 = 输入，0 = 输出）。                                                                 |
| 控制字解析 | 示例 93H = 1001 0011B D7=1（方式控制字）；D6D5=00（A 组方式 0）；D4=1（PA 输入）；D3=0（PC 高 4 输出）；D2=0（B 组方式 0）；D1=1（PB 输入）；D0=1（PC 低 4 输入）。 |

|                                                                                                                          |
|--------------------------------------------------------------------------------------------------------------------------|
| <b>方式 0 编程实例</b>                                                                                                         |
| <ul style="list-style-type: none"><li><b>例 1</b>：8255A 占用 280H-287H 的奇地址，设定 PA 口为输入，PB 口为输出控制 LED 灯的亮灭。总共分为三步：</li></ul> |



|                                                        |
|--------------------------------------------------------|
| <b>工作方式 1 方式 1 输入</b>                                  |
| 将 PC 口征用为握手/中断信号线，A/B 端口用于数据传输，通信通过 STB/IBF/INTR 协同完成。 |

|      |                                                                                         |
|------|-----------------------------------------------------------------------------------------|
| STB  | 方向：外设 → 8255A（输入）。<br>功能：外设准备好数据时发出选通脉冲；STB 的有效边沿（下降沿）用于锁存数据。                           |
| IBF  | 方向：8255A → 外设（输出）。<br>功能：8255A 在锁存后位置，通知外设“缓冲已满，请等待”；CPU 读出数据后由 8255A 清零。               |
| INTR | 方向：8255A → CPU（输出）。<br>功能：当 STB 完成且 IBF 为有效且内部中断使能（INTE=1）时置位，通知 CPU 读取端口数据；CPU 读操作后复位。 |

|              |                                                                            |
|--------------|----------------------------------------------------------------------------|
| <b>A 组映射</b> | PC4=STB_A（输入），PC5=IBF_A（输出），PC3=INTR_A（输出）。                                |
| <b>B 组映射</b> | PC2=STB_B（输入），PC1=IBF_B（输出），PC0=INTR_B（输出）。                                |
| <b>• 注意：</b> |                                                                            |
| <b>中断使能</b>  | 内部的 INTE 为软件可控开关，须由软件置 1 才允许由 INTR 发起中断；即 <b>中断发生需满足 STB、IBF、INTE 条件</b> 。 |

|                                                                                                                                                                                                                                                                                                              |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>方式 1（输入）传输流程</b>                                                                                                                                                                                                                                                                                          |
| <ol style="list-style-type: none"><li>外设准备好数据，向 PC4 送出低电平 STB（选通信号）。</li> <li>8255 在 STB 有效时锁存数据到 PA。</li> <li>8255 将 IBF（PC5）置高，表示缓冲已满并阻止外设再送新数据；同时撤销 STB（变高）。</li> <li>在中断方式下，若 IBF=1 且中断允许，则 PC3 发出 INTR 给 CPU。</li> <li>CPU 响应 INTR 后通过 RD 读出数据。</li> <li>CPU 读出后，8255 将 IBF 清零（置低），准备进入下一轮传送。</li></ol> |

|                                                                                                                                                                                                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>方式 1 输出</b>                                                                                                                                                                                  |
| <div> <div>OBF</div> <div>表示数据已由 CPU 写入 8255，等待外设取走。</div> </div> <div> <div>ACK</div> <div>外设发出的回执信号，表示已取走数据。</div> </div> <div> <div>INTR（中断请求）</div> <div>8255 向 CPU 发出的中断请求信号。</div> </div> |

|                     |                                                                                                                              |
|---------------------|------------------------------------------------------------------------------------------------------------------------------|
| <b>传输流程</b>         | CPU 在写周期（ $\overline{WR}$ 有效）将数据写入 8255 数据端口；8255 发出 OBF（PC7 置低）通知外设取数据；外设取走数据后反馈 ACK（PC6 低电平脉冲）；8255 撤销 OBF（PC7 置高），一次传输完成。 |
| <b>OBF / ACK 含义</b> | OBF（低电平）表示“请外设取数据”；在程序查询方式下亦作为 CPU 查询信号。ACK（低电平脉冲）表示外设已取走数据，向 8255 作应答。                                                      |
| <b>中断方式</b>         | 在中断方式下，OBF 和 ACK 满足规定状态（原文：都为高电平）时，PC3 向 CPU 发出 INTR 信号；CPU 响应后即可开始下一轮数据传输。                                                  |

|                              |
|------------------------------|
| <b>方式 1 编程实例</b>             |
| 这里的代码普遍挺长的，感觉应该不能考，理解工作过程即可。 |
| <b>工作方式 2 • 概述：</b>          |

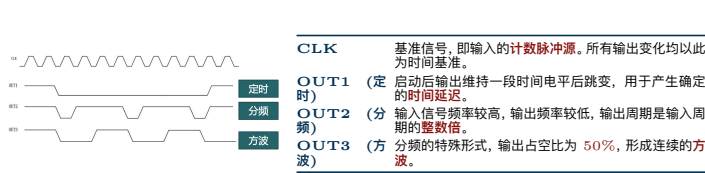
|           |                                                                     |
|-----------|---------------------------------------------------------------------|
| <b>定义</b> | Port A 双向工作（既能输入又能输出），实质上将方式 1 的选通输入与选通输出合并到同一端口，因此也就只需要 PA 端口工作即可。 |
|-----------|---------------------------------------------------------------------|

|             |                                                                                                           |
|-------------|-----------------------------------------------------------------------------------------------------------|
| <b>输入握手</b> | <b>STB（PC4）</b> ：外设选通信号（低有效，外设表示数据已准备好）； <b>IBF（PC5）</b> ：输入缓冲器满（8255 发出，高电平有效），通知外设暂停发送，直至 CPU 取走数据。     |
| <b>输出握手</b> | <b>OBF（PC7）</b> ：输出缓冲器满（8255 发出，低电平有效），表示 CPU 已写入数据； <b>ACK（PC6）</b> ：外设应答（低电平有效），表示外设已取走数据。              |
| <b>公共中断</b> | <b>INTR（PC3）</b> ：输入/输出共用中断请求线。只要满足中断允许条件（INTE=1），输入缓冲器满或输出可接受新数据时均可经 PC3 向 CPU 请求中断；CPU 响应后需查询状态以区分事件类型。 |

由于 PC7、PC3 被 PA 占用，剩余资源为 Port B（8 位）及 PC2、PC0（低 3 位）。Port B 可独立设为方式 0 或方式 1；PC2、PC0 在 B 为方式 1 时作 B 的握手线，否则作普通 I/O 引脚。

**8253 • 概述：** 8253 是一种**可编程的计数器/定时接口芯片**，最高计数频率为 **2MHz**，可用于产生各种**定时波形**，也可用于对外部事件计数。内部有**三个独立的 16 位减一计数器**（互不干扰，支持**二进制（Binary）**或**二十进制（BCD 码）**计数），通过设置控制字，各计数器可以工作于 **6 种工作方式**。

|       |  |
|-------|--|
| 功能简述： |  |
|-------|--|



|                               |                                               |
|-------------------------------|-----------------------------------------------|
| <b>• 系统总线接口（连接 CPU）：</b>      |                                               |
| $\overline{CS} \sim D_0$      | 双向数据总线，用于 CPU 读写控制字或计数值。片选信号，低电平有效，由地址译码电路产生。 |
| $\overline{RD}/\overline{WR}$ | 读/写信号，低电平有效，控制数据传输方向。                         |
| $A_1, A_0$                    | 端口选择：00, 01, 10 分别对应计数器 0、1、2；11 为控制寄存器。      |
| <b>• 计数通道信号（连接外设）：</b>        |                                               |
| $CLK_0 \sim 2$                | 时钟输入，计数脉冲源， <b>下降沿</b> 触发减 1 操作。              |
| $GATE_0 \sim 2$               | 门控输入，用于启动、停止或暂停计数过程。                          |
| $OUT_0 \sim 2$                | 输出信号，计数完成或达到条件时输出特定波形。                        |

- 控制寄存器**：只能进行**写操作**，不能读。CPU 通过向此寄存器写入“控制字”来设定各通道的工作方式。
- 计数通道结构**：包含三个独立的计数通道（0、1、2），内部由以下三部分配合实现“预置 → 减计数 → 读出”逻辑：

**初值寄存器（CR）** **16 位**。用于预置计数的起始值。

**执行部件（CE）** **16 位减法计数器**。在  $CLK$  信号驱动下进行减 1 计数。

**输出锁存器（OL）** **16 位**。用于锁存当前计数值供 CPU 读取，不影响计数进行。

**8253 没有状态寄存器**，CPU **无法直接读取**状态寄存器来获知当前工作状态或回读控制字。这与具有状态回读功能的 8254 不同。

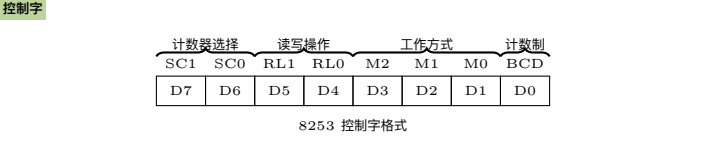
- 端口地址分配**：8253 占用 4 个 I/O 端口，CPU 通过  $A_1, A_0$  选择：

|    |        |
|----|--------|
| 00 | 计数通道 0 |
| 01 | 计数通道 1 |
| 10 | 计数通道 2 |
| 11 | 控制字寄存器 |

|                                                                   |                                                                             |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------|
| <b>• 实例 1：连续地址（8088）：</b>                                         | <b>• 实例 2：偶地址对齐（8086）：</b>                                                  |
| <b>设定</b>                                                         | <b>设定</b>                                                                   |
| 地址范围：0380H ~ 0383H。地址连续（步长为 1）。                                   | 地址序列：0380H, 0382H, ~ 0386H 只使用偶数地址。                                         |
| <b>分析</b>                                                         | <b>分析</b>                                                                   |
| 0380H(…00) → $A_1A_0 = 00$                                        | 0380H(…000) → $A_2A_1 = 00$ ;                                               |
| 0381H(…01) → $A_1A_0 = 01$                                        | 0382H(…010) → $A_2A_1 = 01$ ;                                               |
| 0382H(…10) → $A_1A_0 = 10$                                        | 0384H(…100) → $A_2A_1 = 10$ ;                                               |
| 0383H(…11) → $A_1A_0 = 11$                                        | 0386H(…110) → $A_2A_1 = 11$ 。                                               |
| <b>连接</b>                                                         | <b>连接</b>                                                                   |
| CPU 的 $A_1, A_0$ 直接连接到 8253 的 $A_1, A_0$ 。每一个逻辑地址都对应一个物理端口，无地址间隙。 | CPU $A_0$ 恒为 0。CPU 的 $A_2, A_1$ 错位连接到 8253 的 $A_1, A_0$ 。配合 16 位数据总线字节对齐要求。 |

|                          |                                                         |
|--------------------------|---------------------------------------------------------|
| <b>• 读/写逻辑真值表：</b>       |                                                         |
| <b>片选前</b>               | 所有操作必须在 $\overline{CS} = 0$ 时有效。                        |
| 写（ $\overline{WR} = 0$ ） | 根据 $A_1, A_0$ 写入通道 0/1/2 的计数初值寄存器，或写入控制字寄存器。            |
| 读（ $\overline{RD} = 0$ ） | 根据 $A_1, A_0$ 读取通道 0/1/2 的当前计数值。注意： <b>控制字寄存器只写不读</b> 。 |

- 8 位总线与 16 位计数器接口**：8253 内部的计数初值寄存器（CR）和输出锁存器（OL）都是 16 位的，但外部数据总线（ $D_7 \sim D_0$ ）只有 8 位。必须通过**两次 I/O 操作**来完成一个 16 位数据的传输，由控制寄存器中的控制字来指定读写顺序（如：先读/写低 8 位，再读/写高 8 位）。



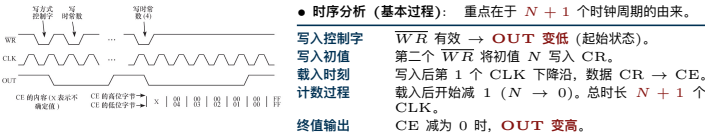
|                                                                                                                                        |                                                                                                                                                                                                  |
|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $SC_1, SC_0$                                                                                                                           | <b>计数器选择</b> 。因共用端口（3 个计数器共用一个控制锁存器），需指定目标通道。<br>00：CNT0；01：CNT1；10：CNT2；11：非法。                                                                                                                  |
| $RL_1, RL_0$                                                                                                                           | <b>读写格式</b> 。定义 CR/OL 的读写位宽及顺序。<br>00：锁存命令（不停止计数读取）；01：仅低 8 位；10：仅高 8 位；11：先后高 8（16 位常用）。                                                                                                        |
| $M_2 \sim M_0$                                                                                                                         | <b>工作方式</b> 。设定方式 0 ~ 5。                                                                                                                                                                         |
| $BCD$                                                                                                                                  | <b>计数制</b> 。0：二进制（FFFFH）；1：BCD 码（9999H）。                                                                                                                                                         |
| <b>• 实例 1：8 位读写模式：</b> 需求：计数器 0，方式 2，仅使用低 8 位，初值 100，二进制计数。地址：70H ~ 73H。                                                               | <b>• 实例 2：16 位读写模式：</b> 需求：计数器 1，方式 1，16 位（先后高），初值 1234，BCD 码。地址：70H ~ 73H。                                                                                                                      |
| <b>SC 00</b> （选择计数器 0）                                                                                                                 | <b>SC 01</b> （选择计数器 1）                                                                                                                                                                           |
| <b>RL 01</b> （只读/写低 8 位）                                                                                                               | <b>RL 11</b> （先低 8 位后高 8 位）                                                                                                                                                                      |
| <b>M 010</b> （方式 2，分频器）                                                                                                                | <b>M 001</b> （方式 1，单稳态）                                                                                                                                                                          |
| <b>BCD 0</b> （二进制计数）                                                                                                                   | <b>BCD 1</b> （BCD 码计数）                                                                                                                                                                           |
| <b>控制字</b> 00010100B = <b>14H</b>                                                                                                      | <b>控制字</b> 01110011B = <b>73H</b>                                                                                                                                                                |
| <b>汇编实现：</b>                                                                                                                           | <b>汇编实现：</b>                                                                                                                                                                                     |
| <ul style="list-style-type: none"><li>MOV AL, 14H</li> <li>OUT 73H, AL；写控制字</li> <li>MOV AL, 100</li> <li>OUT 70H, AL；写低 8 位</li></ul> | <ul style="list-style-type: none"><li>MOV AL, 73H</li> <li>OUT 73H, AL；写控制字</li> <li>MOV AX, 1234H；BCD 码初值</li> <li>OUT 71H, AL；写低 8 位</li> <li>MOV AL, AH</li> <li>OUT 71H, AL；写高 8 位</li></ul> |

- 锁存命令**：解决在计数器运行过程中读取数值不稳定的问题。通过向控制寄存器写入  $RL_1RL_0 = 00$  的控制字，将当前计数值复制到**输出锁存器（OL）**中保持不变，而**执行部件（CE）**继续计数。
- 锁存读出示例：**

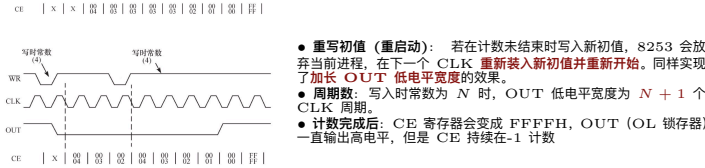
|             |                                                                                                                                                                                                             |
|-------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>锁存机制</b> | 向控制口发送控制字，其中 $RL_1RL_0 = 00$ 。8253 接收后锁存当前值，不影响内部计数。                                                                                                                                                        |
| <b>实例分析</b> | 读取计数器 0 当前值： $SC = 00, RL = 00$ ，其余位无关，控制字为 <b>00H</b> 。                                                                                                                                                    |
| <b>代码流程</b> | <ol style="list-style-type: none"><li>MOV AL, 00H</li> <li>OUT 73H, AL（向控制口发送锁存命令）</li> <li>IN AL, 70H（从数据口读取低 8 位）</li> <li>IN AL, 70H（从数据口读取高 8 位）</li></ol> 读出操作必须符合初始化时设定的 <b>RL</b> 格式（如 16 位模式需连续两次）。 |
| <b>注意</b>   |                                                                                                                                                                                                             |

|                  |                                                                             |
|------------------|-----------------------------------------------------------------------------|
| <b>• 相关名词解释：</b> |                                                                             |
| <b>CLK 脉冲</b>    | 指 <b>CLK</b> 引脚上的信号单元。在计数过程中，每一个 <b>CLK</b> 脉冲的 <b>下降沿</b> 到来时，计数器减 1。      |
| <b>计数器时常数</b>    | 与“计数通道”同义。指通过指令写入计数器的值，等同于 <b>初值</b> 。输出波形的周期或延时由该值决定：时间 = 初值 × $T_{CLK}$ 。 |

|                     |                                                                    |
|---------------------|--------------------------------------------------------------------|
| <b>方式 0 • 功能定义：</b> | 主要用于 <b>定时中断</b> 。给定时间 $t_0$ ，到达后输出信号通知 CPU。                       |
| <b>初始化</b>          | 写入方式控制字后，输出引脚 <b>OUT 变为低电平</b> 。                                   |
| <b>计数过程</b>         | 写入初值后开始减 1 计数，期间 <b>OUT 保持低电平</b> 。                                |
| <b>计数结束</b>         | 当计数值减到 0 时， <b>OUT 立即跳变为高电平</b> 。产生的 <b>上升沿信号</b> 通常连接 CPU 中断请求引脚。 |



|                                                             |                      |
|-------------------------------------------------------------|----------------------|
| <b>• GATE 控制（暂停）：</b>                                       |                      |
| <b>GATE=0</b>                                               | 暂停计数（CE 保持），OUT 保持低。 |
| <b>GATE=1</b>                                               | 恢复计数（从暂停值继续）。        |
| <b>• 结论：</b> 利用 GATE 可 <b>加长 OUT 低电平宽度</b> 。实际时间 = 预置 + 暂停。 |                      |



|                                                              |                                             |
|--------------------------------------------------------------|---------------------------------------------|
| <b>• 方式 0 编程示例：</b> 已知端口 40H ~ 43H，计数器 0，方式 0，初值 1500，二进制计数。 |                                             |
| <b>控制字</b>                                                   | SC=00, RL=11, M=000, BCD=0 → <b>30H</b>     |
| <b>时常数</b>                                                   | 1500 = 05DCH（低位 <b>DCH</b> ，高位 <b>05H</b> ） |
| <b>汇编实现：</b>                                                 |                                             |

|          |                         |
|----------|-------------------------|
| <b>•</b> | MOV DX, 43H；指向控制口       |
| <b>•</b> | MOV AL, 30H；写控制字        |
| <b>•</b> | OUT DX, AL              |
| <b>•</b> | MOV DX, 40H；指向计数器 0 数据口 |
| <b>•</b> | MOV AX, 1500；AX = 05DCH |
| <b>•</b> | OUT DX, AL；写低 8 位（DCH）  |
| <b>•</b> | MOV AL, AH；取高 8 位（05H）  |
| <b>•</b> | OUT DX, AL；写高 8 位       |

- 核心逻辑：** 写初值 → OUT 低 → 计数 → 0 → OUT 高。**定时时长** =  $(N + 1) \times T_{CLK}$ 。可通过 GATE 硬件信号暂停，或重写初值延长定时。
- 方式 1 • 功能定义：** **单脉冲形成**。即产生一个宽度可控的**负脉冲**。

|              |                                                                                 |
|--------------|---------------------------------------------------------------------------------|
| <b>触发源</b>   | <b>硬件触发</b> 。区别于方式 0 的软件写入触发，方式 1 必须由 <b>GATE</b> 引脚的 <b>上升沿</b> （由低电平变高电平）来触发。 |
| <b>脉冲宽度</b>  | 输出负脉冲的宽度为 $N \times T_{CLK}$ ，其中 $N$ 为预置的初值。                                    |
| <b>初始化</b>   | 写入方式控制字后 <b>OUT 变高</b> ；写入初值后 OUT 保持高电平。                                        |
| <b>触发时刻</b>  | <b>GATE 上升沿</b> 触发。在随后的下一个 CLK 下降沿，OUT 由高变低。                                    |
| <b>计数与结束</b> | 计数器减 1 期间 OUT 为低；减至 0 时 OUT 跳变为高。低电平宽度 = $N \times T_{CLK}$ 。                   |

- 可重触发性：** 在脉冲未结束时，若 **GATE** 再次出现**上升沿**，8253 会在下一个 CLK 将初值寄存器（CR）的值**重新装入**执行单元（CE），使计数器重新开始，从而**延长 OUT 低电平宽度**。





● **解（核心计算）**： 方式 3 下输出周期满足：  $T_{\text{out}} = N \cdot T_{\text{clk}}$ 。

$T_{\text{clk}} = 1/f_{\text{in}} = 1\mu\text{s}$ ，故

$$N = \frac{10\text{ ms}}{1\mu\text{s}} = 10000_{10} = 2710_{\text{H}}.$$

端口：CNT0=3F0H, CNT1=3F1H, CNT2=3F2H, CTRL=3F3H。  
控制字：  $SC = 10$ ,  $RL = 11$ ,  $M = 011$ ,  $BCD = 0 \Rightarrow \text{B6H}$ 。

MOV AL, 0B6H ; CNT2, 16 位 (先低后高), 方式 3, 二进制  
OUT 3F3H, AL ; 写控制字到控制口

● **解（核心代码）**：  
MOV AX, 2710H ; N=10000  
OUT 3F2H, AL ; 写低 8 位 10H  
MOV AL, AH  
OUT 3F2H, AL ; 写高 8 位 27H

● **作业 6-2**：假设有一片 8253 芯片，其端口地址为 0FCH 0FFH, 其 CLK 引脚输入的时钟信号周期为 0.84 s。现用该 8253 芯片的计数器 1，工作在方式 2，产生周期为 2ms 的信号，试写出该 8253 芯片的初始化程序。

● **解（核心计算）**： 方式 2:  $T_{\text{out}} = N \cdot T_{\text{clk}}$ 。  $T_{\text{clk}} = 0.84\mu\text{s}$ ,  $T_{\text{out}} = 2\text{ ms}$ ,

$$N = \frac{2\text{ ms}}{0.84\mu\text{s}} \approx 2381_{10} = 094D_{\text{H}}.$$

端口：CNT1=0FDH, CTRL=0FFH; 控制字：  $SC = 01$ ,  $RL = 11$ ,  $M = 010$ ,  $BCD = 0 \Rightarrow 74\text{H}$ 。

MOV AL, 74H ; CNT1, 16 位 (先低后高), 方式 2, 二进制  
OUT 0FFH, AL

● **解（核心代码）**：  
MOV AX, 094DH ; N=2381  
OUT 0FDH, AL ; 低 8 位 4DH  
MOV AL, AH  
OUT 0FDH, AL ; 高 8 位 09H

● **作业 6-3**：假设 8253 各端口的地址分别为 40H 43H. 若计数器 0, OUT 引脚输出频率为 1kHz 的连续方波信号；计数器 1, 每秒产生 18.2 次连续的方波信号；计数器 2, 每 15.12 s 输出一次连续的单脉冲信号。CLK 时钟脉冲的频率为 1.19MHz。对以上计数器通道进行初始化编程。

● **解（核心计算）**： 已知  $f_{\text{clk}} = 1.19\text{ MHz}$ , 则  $T_{\text{clk}} = \frac{1}{f_{\text{clk}}} \approx 0.840\mu\text{s}$ 。

CNT0:  $f_{\text{out}} =$  方式 3:  $f_{\text{out}} = \frac{f_{\text{clk}}}{N} \Rightarrow N = \frac{1.19 \times 10^6}{10^3} = 1190_{10} = 04A6_{\text{H}}$ 。  
1 kHz 方波 (方式 3)

CNT1:  $f_{\text{out}} = N = \frac{1.19 \times 10^6}{18.2} \approx 65385_{10} = FF69_{\text{H}}$  (取最接近整数)。  
18.2 Hz 方波 (方式 3)

CNT2: 周期  $T_{\text{out}} =$  方式 2:  $T_{\text{out}} = N \cdot T_{\text{clk}} \Rightarrow N = \frac{15.12}{0.840} \approx 18_{10} = 0012_{\text{H}}$ 。  
15.12  $\mu\text{s}$  的连续单脉冲 (方式 2)

端口：CNT0=40H, CNT1=41H, CNT2=42H, CTRL=43H。  
控制字：CNT0(方式 3)  $\Rightarrow 36\text{H}$ ; CNT1(方式 3)  $\Rightarrow 76\text{H}$ ; CNT2(方式 2)  $\Rightarrow \text{B4H}$ 。

; CNT0: 1kHz 方波 (Mode 3), N=04A6H

MOV AL, 36H  
OUT 43H, AL  
MOV AX, 04A6H  
OUT 40H, AL  
MOV AL, AH  
OUT 40H, AL

; CNT1: 18.2Hz 方波 (Mode 3), N=FF69H

MOV AL, 76H  
OUT 43H, AL  
MOV AX, 0FF69H  
OUT 41H, AL  
MOV AL, AH  
OUT 41H, AL

; CNT2: 15.12us 周期单脉冲 (Mode 2), N=0012H

MOV AL, B4H  
OUT 43H, AL  
MOV AX, 0012H  
OUT 42H, AL  
MOV AL, AH  
OUT 42H, AL

● **作业 6-4**：假设 8253 各端口的地址分别为 40H 43H, CLK 时钟信号频率为 1MHz, 若要计数器产生周期为 1s 的方波, 请问使用一片 8253 如何达到目的? 写出初始化程序。

● **解(核心思路)**： 单计数器最大初值仅 65536, 在  $f_{\text{clk}} = 1\text{ MHz}$  下无法直接得到 1 s 周期(需分频  $N = 10^6$ )。

用一片 8253 内部两级级联分频: OUT0  $\rightarrow$  CLK1。

取两级均为方式 3 (方波), 令  $N_0 = N_1 = 1000$ , 则总分频 =  $1000 \times 1000 = 10^6$ , 最终得到 1 Hz 方波 (周期 1 s)。

● **解（核心计算）**：

$$N = \frac{f_{in}}{f_{out}} = \frac{10^6}{1} = 10^6, \quad 10^6 = 1000 \times 1000$$

$$1000_{10} = 03E8_{\text{H}}$$

控制字 (均为方式 3、16 位先低后高、二进制计数)：

CNT0:  $SC = 00$ ,  $RL = 11$ ,  $M = 011 \Rightarrow 36\text{H}$ ; CNT1:  $SC = 01$ ,  $RL = 11$ ,  $M = 011 \Rightarrow 76\text{H}$ 。

; CNT0: Mode 3, NO=1000 = 03E8H (输出 1kHz 方波)

MOV AL, 36H  
OUT 43H, AL  
MOV AX, 03E8H  
OUT 40H, AL  
MOV AL, AH  
OUT 40H, AL

● **解（核心代码）**： ; 硬件: OUT0  $\rightarrow$  CLK1

; CNT1: Mode 3, N1=1000 = 03E8H (输出 1Hz 方波, T=1s)

MOV AL, 76H  
OUT 43H, AL  
MOV AX, 03E8H  
OUT 41H, AL  
MOV AL, AH  
OUT 41H, AL

● **作业 6-5**: 假设 8255A 芯片起始端口地址为 60H, 编写指令序列分别完成下列配置：

● (1): A/B 组方式 0; PB、PC 输出; PA 输入  
● (2): A 组方式 2; B 组方式 0; P/B 输出  
● (3): A 组方式 1 (PA 输入、PC7/PC6 输出); B 组方式 1 (PB 输入)

| 控制字 | 90H                        | 控制字                         | 0C0H                        | 控制字                         | 0B6H                        |
|-----|----------------------------|-----------------------------|-----------------------------|-----------------------------|-----------------------------|
| 代码  | MOV AL, 90H<br>OUT 63H, AL | MOV AL, 0C0H<br>OUT 63H, AL | MOV AL, 0C0H<br>OUT 63H, AL | MOV AL, 0B6H<br>OUT 63H, AL | MOV AL, 0B6H<br>OUT 63H, AL |

● **作业 6-6**： 现要求用一个 8255A 作为终端机的接口。由 PA 通道输出字符到终端机的显示缓冲器, PB 通用于键盘输入字符, PC 通道为终端状态信息输入通道。当 PC0=1 表示键盘输入字符就绪, PC7=0 表示显示缓冲器已空。要求用查询方法把从键盘输入的每个字符都送到终端机的显示缓冲器上, 当输入的是回车符时 (ASCII 码为 0DH) 则操作结束。假设该 8255A 芯片的端口地址为 60H~63H, 请编写包括 8255A 初始化的输入/输出驱动程序

● **题目分析**:

|                             |                                          |
|-----------------------------|------------------------------------------|
| PA <input type="checkbox"/> | 输出字符到显示缓冲器 (输出)                          |
| PB <input type="checkbox"/> | 从键盘接收字符 (输入)                             |
| PC <input type="checkbox"/> | 读取状态信息 (输入); PC0=1: 键盘就绪; PC7=0: 显示缓冲器已空 |
| 通信方式                        | 查询 (轮询) 方式, 监测 PC0 直到获得就绪信号              |
| 终止条件                        | 输入的字符为回车符 (0DH) 时程序结束                    |

● **初始化设置**:

|                             |                      |
|-----------------------------|----------------------|
| PA <input type="checkbox"/> | 方式 0、输出 (D4=0)       |
| PB <input type="checkbox"/> | 方式 0、输入 (D1=1)       |
| PC <input type="checkbox"/> | 方式 0、输入 (D0=1, D3=0) |
| 控制字值                        | 10001011B = 8BH      |

| 地址定义           | 说明                                    |
|----------------|---------------------------------------|
| COUNTA EQU 60H | PA <input type="checkbox"/> 地址 (键盘输出) |
| COUNTB EQU 61H | PB <input type="checkbox"/> 地址 (键盘输入) |
| COUNTC EQU 62H | PC <input type="checkbox"/> 地址 (状态输入) |
| COUNTD EQU 63H | 控制口地址                                 |

● **汇编程序**:

| 指令段                                                               | 说明                                                                                             |
|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------|
| <b>8255A 初始化</b>                                                  |                                                                                                |
| MOV AL, 8BH<br>MOV DX, COUNTD<br>OUT DX, AL                       | 控制字: PA 输出, PB/PC 输入<br>加载控制口地址<br>写入控制字                                                       |
| <b>主循环: 接收键盘输入</b>                                                |                                                                                                |
| AGAIN:<br>MOV DX, COUNTC<br>IN AL, DX<br>TEST AL, 01H<br>JZ AGAIN | 循环标号<br>指向 PC <input type="checkbox"/> (状态口)<br>读取状态信息<br>检测 PC0 位 (键盘输入就绪)<br>若 PC0=0, 继续查询等待 |
| <b>读取并处理字符</b>                                                    |                                                                                                |
| MOV DX, COUNTB<br>IN AL, DX<br>CMP AL, 0DH<br>JZ DONE             | 指向 PB <input type="checkbox"/> (键盘输入)<br>从键盘读取字符<br>与回车符 (0DH) 比较<br>若是回车符, 程序结束               |
| <b>输出到显示缓冲器</b>                                                   |                                                                                                |
| MOV DX, COUNTA<br>OUT DX, AL<br>JMP AGAIN                         | 指向 PA <input type="checkbox"/> (显示输出)<br>将字符输出到显示<br>返回, 接收下一个字符                               |
| <b>程序结束</b>                                                       |                                                                                                |
| DONE:<br>RET                                                      | 结束标号<br>返回调用程序                                                                                 |